Ultra-Detailed Week 1 DSA Mastery Guide

Target: FAANG/Product-Based Senior Engineer Interviews **Focus Topics:** Arrays, Hashing, Two-Pointers, Advanced Math **Core References:** TUF (Striver), Cracking the Coding Interview

---

Table of Contents

1. Day 1: Arrays - Matrices & Logic
2. Day 2: Arrays - Pre-computation & Operations
3. Day 3: Arrays - Rotations & Merging
4. Day 4: Hashing & Two Pointers Fundamentals
5. Day 5: Hashing & Prefix Sum Advanced
6. Day 6: Arrays - Math & Majority Elements
7. Day 7: Two Pointer Advanced & Architectural Arrays
8. Weekly Revision Cheatsheet

Day 1: Arrays - Matrices & Logic

Welcome to Day 1 of your DSA Masterclass! As a Principal Engineer, I am not here to just teach you code; I am here to teach you **how to think**. Product-based companies (FAANG) don't just want working code—they want developers who understand memory, time-space trade-offs, and state management. Today, we conquer 2D Arrays (Matrices) and Array Logic permutations from the TUF (Striver) SDE Sheet.

🔥 Problem 1: Set Matrix Zeroes

🔹 0. Company Tagging

- **Asked in:** Amazon, Microsoft, Meta
- **Frequency:** 🔥 High
- **Interview Context:** Why do interviewers ask this? It tests your ability to incrementally optimize **Space Complexity**. Anyone can solve it with `O(M x N)` extra space. A Senior Engineer solves it in `O(1)` space by using the data structure *itself* as memory storage.

🔹 1. Problem Explanation

Given an `m x n` integer matrix, if an element is `0`, set its entire row and column to `0`'s. You must do it **in-place**.

- **Real-world analogy:** Imagine a spreadsheet of server statuses. If one server goes down (status `0`), the entire network rack (row) and power grid line (column) MUST be marked as inactive.
- **Input:** `matrix = [[1,1,1],[1,0,1],[1,1,1]]`
- **Output:** `[[1,0,1],[0,0,0],[1,0,1]]`
- **Constraints:** You cannot modify the matrix sequentially while scanning, otherwise a cascading effect will turn the *entire* matrix to 0s instantly!

🔹 2. Data Structure Masterclass

- **DS Used:** 2D Array / Matrix.
- **Why chosen:** Hardware architecture stores 2D arrays in contiguous memory (Row-Major format).
- **Time complexity benefits:** Scanning by rows benefits from CPU Cache Spatial Locality (`O(1)` cache hits).
- **Trade-offs:** 2D arrays are rigid in size. Inserting/Deleting rows is disastrous (`O(M x N)` shifting).

🔹 3. Pattern Recognition

- **Pattern name:** Dummy Array State Tracking / In-place Hashing.
- **WHEN to use:** When you need to track state changes but modifying the array sequentially ruins the original state.
- **HOW to identify:** Keywords like "Matrix", "Set entire row/col", "In-place".

🔹 4. Solution Evolution

✅ Approach 1: Brute Force

- **Thinking:** I'll duplicate the entire matrix. I'll scan the original. If I see a `0`, I'll write `0`s to the duplicate matrix's row and col. Copy the duplicate back at the end.
- **Code:** Too trivial to write out.
- **Complexity:** Time `O(N x M x (N + M))`, Space `O(N x M)` (Full matrix copy).
- **Why it's bad:** Disastrous memory allocation. If the matrix is 10,000x10,000, you just blew up the JVM Heap.

✅ Approach 2: Better Approach (Dummy Arrays)

- **Idea:** We don't need a whole matrix to remember what to zero out. We just need two boolean lists: one for rows, one for columns.
- **Code:**

```java
public void setZeroesBetter(int[][] matrix) {
    int rows = matrix.length;
    int cols = matrix[0].length;

    int[] dummyRow = new int[rows];
    int[] dummyCol = new int[cols];

    // Pass 1: Tag the rows and cols that need to be zeroed
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            if (matrix[i][j] == 0) {
                dummyRow[i] = 1;
                dummyCol[j] = 1;
            }
        }
    }

    // Pass 2: Actually set the zeroes based on our dummy arrays
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            if (dummyRow[i] == 1 || dummyCol[j] == 1) {
                matrix[i][j] = 0;
            }
        }
```

```
    }
}
```

- **Improvement:** Space drops from `O(N x M)` to `O(N + M)` ! Time is strictly `O(N x M)` . This is good enough to pass most interviews, but a Senior aims for perfection.

✅ Approach 3: Optimal Approach (`O(1)` Space)

- **Best solution idea:** The interviewer explicitly asks for `O(1)` space. Where do we find `O(N+M)` free memory? **Inside the matrix itself!**
  - We use the *first row* and *first column* of the matrix as our `dummyRow` and `dummyCol` .
  - **The Catch:** The first row and first column intersect at `matrix[0][0]` . We'd have a collision!
  - **The Fix:** `matrix[0][0]` tracks the 0th row. We create exactly *one extra variable* `col0` to track the 0th column.
- **Optimal Java Code:**

```java
public class SetMatrixZeroes {
    public void setZeroesOptimal(int[][] matrix) {
        int col0 = 1; // 1 means safe, 0 means zero-out the 0th column
        int rows = matrix.length;
        int cols = matrix[0].length;

        // Step 1: Scan and tag state in the first row / col
        for (int i = 0; i < rows; i++) {
            if (matrix[i][0] == 0) col0 = 0; // Mark the special col0 tracker
            for (int j = 1; j < cols; j++) {
                if (matrix[i][j] == 0) {
                    matrix[i][0] = 0; // Mark the row head
                    matrix[0][j] = 0; // Mark the col head
                }
            }
        }

        // Step 2: Mutate the matrix from BOTTOM-RIGHT to TOP-LEFT
        // We traverse backward so we don't accidentally overwrite our row/col
headers too early!
        for (int i = rows - 1; i >= 0; i--) {
            for (int j = cols - 1; j >= 1; j--) {
                // If either the row head or col head is 0, make it 0
                if (matrix[i][0] == 0 || matrix[0][j] == 0) {
```

```
                    matrix[i][j] = 0;
                }
            }
            // Finally, process the 0th column separately using col0 variable
            if (col0 == 0) {
                matrix[i][0] = 0;
            }
        }
    }
}
```

5. Dry Run

Let `matrix = [[1, 1, 1], [1, 0, 1], [1, 1, 1]]`

1. **P1 (i=0):** `matrix[0][0]` !=0. `col0=1`.
2. **P1 (i=1):** `matrix[1][0]` !=0. `j=1`: Ah! `matrix[1][1]` is 0!
   - Tag row: `matrix[1][0] = 0`. Tag col: `matrix[0][1] = 0`.
3. **P1 (i=2):** No zeros.
4. Current Matrix headers: Row 0 is `[1, 0, 1]`. Col 0 is `[1, 0, 1]`. `col0=1`.
5. **P2 (Backward Scan):**
   - We hit `i=2, j=2`: checks heads, both 1. Leaves it.
   - We hit `i=1, j=1`: checks `matrix[1][0] == 0`. Sets it to 0. (Correct!)
   - At the very end, `col0` is 1, so the 0th column is left alone (Correct!).

6. Architectural Thinking

- **WHAT:** Using data structure boundaries as caching layers.
- **WHERE:** Flow moves from establishing invariants (headers) to aggressively mutating downward dependent nodes.
- **HOW:** Backward scanning forces dependencies to resolve perfectly, preventing state-corruption race conditions!
- **WHO:** This pattern of "stealing the first 64 bytes for header metadata" is exactly how Network Packets and OS Page Tables work. Software architecture mimics low-level hardware memory layouts!

7. Edge Cases

- **Completely empty matrix (`[]`):** Null check needed in production.
- **First cell `[0][0]` is already 0:** Setting `col0=0` gracefully handles it without blowing up logic.

- **Matrix consisting of only one row / one col:** Handled perfectly by loop bounds.

🔹 8. Clean Code (Java Best Practices)

- Single variable definitions (`int rows` , `int cols`) prevent recalculating `.length` arrays within tight nested loops. JVM Hotspot might optimize it, but explicit scoping shows architectural intent.

🔹 9. Interview Training

- **How to explain:** "A brute force copy takes `O(MN)` space. We can flatten the state into two boundary arrays of size `M` and `N` . But to go `O(1)` space, I want to hijack the first row and column of the matrix itself to act as my state arrays, using one primitive integer to resolve the corner collision."
- **Mistakes to avoid:** Do NOT scan Phase 2 from top-left to bottom-right! You will erase your own header tags instantly. Scan backwards.

🔥 Problem 2: Pascal's Triangle

🔹 0. Company Tagging

- **Asked in:** Google, Amazon
- **Frequency:** ⚡ Medium
- **Interview Context:** This isn't just about loops. The interviewer is testing if you recognize **Combinatorics Math** (nCr). An amateur uses additive logic (`row[i] = prev[i] + prev[i-1]`). A master uses Math permutations inside an `O(N^2)` algorithm for blazingly fast pure generation.

🔹 1. Problem Explanation

Given an integer `numRows` , generate the first `numRows` of Pascal's triangle. In Pascal's triangle, each number is the sum of the two numbers directly above it.

- **Output (numRows=5):** `[[1], [1,1], [1,2,1], [1,3,3,1], [1,4,6,4,1]]`

There are three variations they could ask:

1. Print the exact element at row `R` and col `C` .
2. Print the entire `R-th` row.
3. Print the entire triangle.

We will focus on the ultimate generation (Variation 3).

🔹 2. Data Structure Masterclass

- **DS Used:** `List<List<Integer>>` (Dynamic jagged arrays).
- **Why chosen:** Java arrays are fixed size. Pascal's triangle rows grow by 1 each iteration. `ArrayList` is required.
- **Time complexity benefits:** `ArrayList` appends are amortized `O(1)`.

3. Pattern Recognition

- **Pattern name:** Dynamic Programming (Pre-computation) OR Combinatorics (`nCr`).
- **WHEN to use:** Generating sequences where $State_{N} `strictly relies on` State_{N-1}$.

🔹 4. Solution Evolution

✅ Approach 1: Additive Method (Standard / Better)

- **Idea:** Just mimic the definition. Row 5 is built by looking at Row 4. Loop through previous row, add adjacent elements.
- **Code:**

```java
public List<List<Integer>> generate(int numRows) {
    List<List<Integer>> result = new ArrayList<>();
    if (numRows == 0) return result;

    result.add(new ArrayList<>());
    result.get(0).add(1); // Row 1

    for (int i = 1; i < numRows; i++) {
        List<Integer> prevRow = result.get(i - 1);
        List<Integer> currRow = new ArrayList<>();

        currRow.add(1); // First element is always 1
        for (int j = 1; j < i; j++) {
            currRow.add(prevRow.get(j - 1) + prevRow.get(j));
        }
        currRow.add(1); // Last element is always 1

        result.add(currRow);
    }
    return result;
}
```

- **Complexity:** Time `O(N^2)`, Space `O(N^2)` to hold the triangle. This is totally valid and expected!

✅ Approach 2: Optimal Row Generation (The Mathematical Approach)

- **Idea:** What if the interviewer says: "Given `N = 100`, just print the 100th row. Do not generate the first 99 rows."
- **The Magic:** Any cell in Pascal's Triangle is exactly given by the permutation formula: `nCr` (n Choose r).
  - Row 4 is: `^4C_0`, `^4C_1`, `^4C_2`, `^4C_3`, `^4C_4`
  - We can generate the *entire row* mathematically in `O(N)` time, skipping all previous rows!
  - `nCr` formula simplified trick: `ans = ans * (row - col) / col`
- **Optimal Math Code (Generates 1 single row instantly):**

```java
public List<Integer> generateRow(int row) {
    long ans = 1; // Use long to prevent massive permutation overflow
    List<Integer> ansRow = new ArrayList<>();
    ansRow.add(1); // 0th element is 1

    // Formula: For row=5, generating dynamically:
    // element 1: 1 * (5-1) / 1 = 4
    // element 2: 4 * (5-2) / 2 = 6
    for (int col = 1; col < row; col++) {
        ans = ans * (row - col);
        ans = ans / col;
        ansRow.add((int)ans);
    }
    return ansRow;
}
```

- **Putting it together for the Full Triangle:** You just loop `0` to `numRows` calling `generateRow(i)`. Both approaches yield `O(N^2)` time. But the Math approach requires strictly zero memory lookups!

🔹 5. Dry Run (Math Approach for Row 5)

- Row=5. First element = `1`.
- Col=1: `ans = 1 * (5-1) / 1 = 4`. Row = `[1, 4]`
- Col=2: `ans = 4 * (5-2) / 2 = 6`. Row = `[1, 4, 6]`
- Col=3: `ans = 6 * (5-3) / 3 = 4`. Row = `[1, 4, 6, 4]`

- Col=4: `ans = 4 * (5-4) / 4 = 1`. Row = `[1, 4, 6, 4, 1]`. Correct!

🔹 6. Architectural Thinking

- **WHAT:** Decoupling dependent algorithms into independent mathematical assertions.
- **WHERE:** We abstracted row-calculations away from State-Tracking (the DP method).
- **HOW:** Math formulas (`nCr`) operate without Context (without needing the prior row).
- **WHO:** Stateful APIs rely on caching. Stateless APIs calculate via pure math functions. The Math Approach is identical to modern Stateless Lambda functions in distributed environments!

🔹 7. Edge Cases

- Huge row calculations > ~row 35 will overflow integer boundaries. Moving to `long` is mandated, and for truly huge numbers, Java `BigInteger` is needed. Explain this explicitly to the interviewer.

🔹 8. Clean Code

- When writing math-bound logic, avoid zero-division. We start the `col` iteration at `1` specifically to avoid `col=0` division explosions.

🔹 9. Interview Training

- Provide the Additive Approach first. Usually, that's what they want.
- If asked "Give me row 50 only, space optimized", immediately deploy the `nCr` formula loop. It obliterates the competition.

🔥 Problem 3: Next Permutation

🔹 0. Company Tagging

- **Asked in:** Google, Meta, Bloomberg
- **Frequency:** 🔥 High
- **Interview Context:** Why? This tests whether you "know the trick" or if you can logically deduce suffix patterns. If you attempt brute force permutations (generating all factorial $N!$ states), you immediately signal junior-level thinking.

🔹 1. Problem Explanation

Given an array representing a number (`[1, 2, 3]`), find the numerically "next" largest permutation (`[1, 3, 2]`). If it's already the highest possible configuration (`[3, 2, 1]`), reset it back to the lowest (`[1, 2, 3]`).

- **Analogy:** A combination lock spinning one definitive "click" upwards in lexical dictionary order.

🔹 2. Data Structure Masterclass

- **DS Used:** Array.
- **Why chosen:** Required for element swapping. Memory efficiency relies *heavily* on tracking index points without making substring copies.

🔹 3. Pattern Recognition

- **Pattern name:** Lexicographical Peak Finding (Lexical Suffix algorithm).
- **WHEN to use:** Anytime the word "dictionary order" or "next permutation" appears.

🔹 4. Solution Evolution

✅ Approach 1: Brute Force

- **Idea:** Recursively generate all permutations of the array. Sort them in ascending order. Find where the given input is, output the one element right after it.
- **Complexity:** Generating permutations takes `O(N! x N)` time. For `N=20`, that's $20!$ (quintillions of operations). The universe will end before the test case finishes.

✅ Approach 2 & 3: Optimal (Direct Math Sweep)

- **Visualizing the math:** Look at `2 1 5 4 3 0 0`. What is the NEXT larger number?
- We naturally read numbers left to right, but changes must happen right to left.
- Look from the right: `0, 0, 3, 4, 5` ... that sequence is strictly increasing (right-to-left). A sequence in decreasing order (left-to-right) is at its MAXIMUM possible value. It cannot be increased further!
- We find the first digit that breaks this rule ("The Breakpoint"). Here, moving right-to-left: `5` > `4` > `3` > `0` > `0`, suddenly we hit `1`! Ah! `1 < 5`. The `1` is the bottleneck.
- **The Algorithm:**
  1. Scan R-to-L. Find first `arr[i] < arr[i+1]`. This is the `breakPoint`.
  2. If there is no breakPoint, the array is `[5,4,3,2,1]`. Simply reverse it to `[1,2,3,4,5]` and exit.
  3. Scan R-to-L again. Find the first element strictly larger than `arr[breakPoint]`.
  4. Swap them.
  5. The suffix to the right of the breakPoint will now be in perfectly descending order. To make it the *smallest* possible "next" sequence, simply reverse the suffix to ascending order!

- **Clean Java Code:**

```java
public class NextPermutation {
    public void nextPermutation(int[] nums) {
        if (nums == null || nums.length <= 1) return;

        int i = nums.length - 2;
        // 1. Find the breakpoint
        while (i >= 0 && nums[i] >= nums[i + 1]) {
            i--;
        }

        // 2. If breakpoint exists, find the swapper and swap
        if (i >= 0) {
            int j = nums.length - 1;
            while (nums[j] <= nums[i]) {
                j--;
            }
            swap(nums, i, j);
        }

        // 3. Reverse everything from the breakpoint + 1 to the end
        // If array was fully maxed (i = -1), this reverses the entire array
        reverse(nums, i + 1, nums.length - 1);
    }

    private void swap(int[] nums, int i, int j) {
        int temp = nums[i];
        nums[i] = nums[j];
        nums[j] = temp;
    }

    private void reverse(int[] nums, int start, int end) {
        while (start < end) {
            swap(nums, start, end);
            start++;
            end--;
        }
    }
}
```

5. Dry Run

Input: `nums = [1, 3, 5, 4, 2]`

- **Step 1:** Right loop. `2` < `4` (No). `4` < `5` (No). `5` > `3` (YES!). Breakpoint `i=1` (value `3`).
- **Step 2:** Right loop again to find `>3`. `2` > 3 (No). `4` > 3 (YES). Swap index `j=3` (value `4`).
- **Swap:** Swap index 1 and 3. Array becomes `[1, 4, 5, 3, 2]`.
- **Reverse:** Reverse everything right of index 1 (the suffix `[5, 3, 2]`). It becomes `[2, 3, 5]`.
- **Result:** `[1, 4, 2, 3, 5]`. Exact dictionary next!

🔹 6. Architectural Thinking

- **WHAT:** Operating on suffixes based on numerical polarity.
- **WHERE:** Utilizing the right-hand (least-significant) side of the integer constraint.
- **HOW:** Swapping the minimal upgrade digit and wiping the trailing digits to zero-state (ascending).
- **WHEN:** In precisely three linear sweeps.
- **WHO:** This permutation generation is exactly how C++ `std::next_permutation` is written natively in hardware compilers!

🔹 7. Edge Cases

- **Duplicate Numbers (`[1, 5, 1]`).** Standard `<` operators fail in duplicates. You MUST use `>=` in `while(nums[i] >= nums[i+1])` to gracefully step past grouped dupes.
- **Already maximum (`[3, 2, 1]`).** The `i >= 0` check protects the swap. It skips right into reversing the array from index `0`. Perfectly handles cyclic arrays.

🔹 8. Clean Code

- By extracting `swap` and `reverse` into extremely tight helper variables, the structural code reads exactly like the 3-step proof. Always modularize operations inside complex loop bounds!

🔹 9. Interview Training

- **How to explain:** Start immediately by drawing out a number sequence on the whiteboard. "If we have `1432`, we read backwards to find the drop. `2` to `3` is smooth. `3` to `4` is smooth. `4` dropping to `1` is the bottleneck..." This shows genuine comprehension, not just memorized code.

📝 DAILY SUMMARY

🎯 Patterns Learned Today

1. **State Matrix Boundary Hashing:** Using `matrix[0][n]` and `matrix[m][0]` as bit-flags to solve 2D mapping problems in strictly `O(1)` space.
2. **Combinatorics Magic:** Memorizing the `nCr` DP formula transforms matrix DP mapping from an intensive loop algorithm into a purely stateless mathematical calculation.
3. **Lexicographical Sequence Manipulation:** To augment dictionary combinations, isolate the longest descending suffix, swap the node just prior with its closest larger relative, and reverse the suffix.

⚖️ Decision Rules

- If given a **Matrix** with state-flag requirements and an `O(1)` space limit $=>$ Exploit the first row and column natively.
- If asked for **Combinations/Pascal** specifically targeting one deep row $=>$ Use pure math (`ans = ans * (row - col) / col`).
- If given a numbers-array and asked for **"Next Order"** $=>$ Right-to-Left suffix algorithm (Never Bruite force factorials).

⚠️ Key Mistakes to Avoid

- **Set Matrix Zeroes:** Setting the first row/column header markers in Phase 1, and then in Phase 2, accidentally scanning Top-Left to Bottom-Right. The top-left `0` will overwrite your column markers instantly before you even read them! *Always scan Phase 2 from Bottom-Right to Top-Left.*
- **Next Permutation:** Forgetting the `=` sign in `>=`. Array duplicates will infinite-loop or logic-fail.

See you tomorrow for Day 2: Array Sums & Kadane's!

Day 2: Arrays - Pre-computation & Operations

Welcome to Day 2! Today we focus on algorithms that rely on **Pre-computation and State Carry-over**. In Junior development, arrays are scanned redundantly `O(N^2)` times. In Senior development, arrays are scanned exactly once `O(N)` by carrying the optimal history forward. We will master Kadane's Algorithm, Dijkstra's 3-Way Partitioning, and localized difference tracking.

🔥 Problem 1: Maximum Subarray Sum (Kadane's Algorithm)

🔹 0. Company Tagging

- **Asked in:** Amazon, Microsoft, LinkedIn, Apple
- **Frequency:** 🔥 High
- **Interview Context:** This is the quintessential Dynamic Programming (DP) question masked as an Array question. Interviewers use it to see if you intuitively understand how to aggressively discard toxic data (negative sums) to optimize future sequences.

🔹 1. Problem Explanation

Given an integer array `nums` containing both positive and negative integers, find the contiguous subarray (containing at least one number) which has the largest sum, and return its sum.

- **Real-world analogy:** Tracking the highest net-profit stretch for a volatile stock. If a sequence of days loses you so much money that your net running total goes net-negative, that specific stretch is mathematically poisoned. It can *never* help a future sequence reach a high score.
- **Input:** `nums = [-2, 1, -3, 4, -1, 2, 1, -5, 4]`
- **Output:** `6` (from subarray `[4, -1, 2, 1]`)
- **Constraints:** Must be `O(N)` time.

🔹 2. Data Structure Masterclass

- **DS Used:** Primitive track variables (`maxSum`, `currentSum`).
- **Why chosen:** Standard 1D DP requires an entire `O(N)` array to track history. But since Kadane's only relies on the strictly *immediate preceding* state, we can compress the `O(N)` DP array into a single `O(1)` primitive integer.
- **Trade-offs:** We lose the ability to lookup the optimal sequence sum at *any* arbitrary past index, but we save exponential memory allocation.

🔹 3. Pattern Recognition

- **Pattern name:** Kadane's Algorithm (Greedy State Resetting).
- **WHEN to use:** When asked for maximum/minimum contiguous sums/products in arrays with negative numbers.
- **HOW to identify:** Keywords like "largest sum", "contiguous", "subarray", "contains negatives".

🔹 4. Solution Evolution

✅ Approach 1: Brute Force

- **Thinking:** I'll check every single possible subarray block. Start at `0`, sum to `N`. Start at `1`, sum to `N`. Track maximum.
- **Complexity:** Time `O(N^2)`, Space `O(1)`.
- **Why it's bad:** Time Limit Exceeded (TLE) on arrays larger than `10^5`. Redundantly calculating `sum(A..C)` when you already knew `sum(A..B)` in the previous loop iteration.

✅ Approach 2: Better Approach

- Technically, the `O(N^2)` prefix-sum nested loop *is* the better approach compared to the catastrophic `O(N^3)` triple loop. But there is no real middle-ground pattern here. You either know Kadane's or you fail the time constraints.

✅ Approach 3: Optimal Approach (Kadane's)

- **Best solution idea:** Carry a running sum. Ask one simple question at every step: **"Does adding my running sum to the current element make it bigger, or is my running sum so toxic (negative) that the current element is better off starting a brand new sequence by itself?"**
- **Optimal Java Code:**

```java
public class MaximumSubarray {
    public int maxSubArray(int[] nums) {
        // Initialize max extremely small to handle arrays with ALL negative
numbers
        int maxSum = Integer.MIN_VALUE;
        int currentSum = 0;

        for (int num : nums) {
            // 1. Add current element to the running state
            currentSum += num;

            // 2. Did we achieve a new global high score?
            if (currentSum > maxSum) {
                maxSum = currentSum;
            }

            // 3. The Greedy Cut-off: If our running total dropped below zero,
```

```
            // it is mathematically useless for the future. Reset it.
            if (currentSum < 0) {
                currentSum = 0;
            }
        }

        return maxSum;
    }
}
```

5. Dry Run

Input: `[-2, 1, -3, 4, -1, 2, 1, -5, 4]` // Initial: `max = MIN`, `curr = 0`

1. `num = -2` : `curr` =-2. `max` =-2. `curr < 0` $=>$ `curr=0`. (Drop the toxic past)
2. `num = 1` : `curr` =1. `max` =1.
3. `num = -3` : `curr` =-2. `max` =1 (unchanged). `curr < 0` $=>$ `curr=0`.
4. `num = 4` : `curr` =4. `max` =4.
5. `num = -1` : `curr` =3. `max` =4. (Notice we keep the 3! A minor dip is fine as long as `curr` > 0).
6. `num = 2` : `curr` =5. `max` =5.
7. `num = 1` : `curr` =6. `max` =6. (New Peak!)
8. `num = -5` : `curr` =1. `max` =6.
9. `num = 4` : `curr` =5. `max` =6. Result: `6`.

6. Architectural Thinking

- **WHAT:** Aggressive sequence termination constraint.
- **WHERE:** Executed continuously across linear data streaming.
- **HOW:** By severing the historical payload (resetting to 0) the moment the aggregated weight mathematically harms the possibility of future growth.
- **WHEN:** Checked strictly *after* evaluating if the current dip was part of a new high score.
- **WHO:** This principle is identical to TCP packet windowing resets when congestion drops below acceptable thresholds!

7. Edge Cases

- **All Negative Numbers (`[-5, -2, -9]`):** If you initialized `maxSum = 0`, your logic would return `0` (wrong!). Initializing to `Integer.MIN_VALUE` guarantees it correctly identifies `-2` as the maximum possible subarray (size 1).

🔹 8. Clean Code

- Maintain the strict ordering of the internal `if` statements. Adding to `currentSum`, checking `maxSum`, and *then* aggressively resetting `currentSum < 0` is the most resilient, cleanly readable format.

🔹 9. Interview Training

- **Follow-up Question:** "What if I want the START and END indices of the subarray, not just the sum?"
- **Senior Answer:** "I will add a `tempStart` variable initialized to 0. Whenever `currentSum` resets to 0, I set `tempStart = i + 1`. Whenever `currentSum > maxSum`, I update the final `globalStart = tempStart` and `globalEnd = i`. This tracks the physical bounds effortlessly in `O(1)` space."

🔥 Problem 2: Sort Colors (0s, 1s, and 2s)

🔹 0. Company Tagging

- **Asked in:** Microsoft, Amazon, Adobe
- **Frequency:** ⚡ Medium
- **Interview Context:** This problem tests if you can implement the **Dutch National Flag (DNF)** algorithm. If you sort the array (`O(N log N)`), you fail the space/time check. If you count frequencies and overwrite (`O(2N)`), you pass, but the interviewer will push for a strict 1-Pass `O(N)` solution.

🔹 1. Problem Explanation

Given an array containing only `0`, `1`, and `2`, sort them in-place so all 0s are first, 1s in the middle, and 2s at the end.

- **Real-world analogy:** An assembly line sorting defective objects (0), acceptable objects (1), and premium objects (2). You only have one robotic arm and must push items to their respective zones in a single sweep without additional bins.
- **Constraints:** Solve in exactly 1 pass (`O(N)` time), `O(1)` space.

🔹 2. Data Structure Masterclass

- **DS Used:** Array (In-place pointers).
- **Why chosen:** Required by problem for `O(1)` modifications.

- **Internal working:** Using 3 pointers to define rigid territory boundaries inside the contiguous memory block.

🔹 3. Pattern Recognition

- **Pattern name:** Dijkstra's 3-Way Partitioning (Dutch National Flag).
- **WHEN to use:** When categorically segregating elements into exactly 3 groups.
- **HOW to identify:** Keywords "sort 0 1 2", "three colors", "one pass".

🔹 4. Solution Evolution

✅ Approach 1: Counting Sort (Better)

- **Thinking:** Two passes. Pass 1: count how many 0s, 1s, and 2s exist. Pass 2: Overwrite the array from index `0` sequentially based on the counts.
- **Complexity:** Time `O(2N)`, Space `O(1)`. Good, but not 1 pass.

✅ Approach 3: Optimal Approach (Dutch National Flag)

- **Best solution idea:** Use 3 pointers (`low`, `mid`, `high`).
  - `0` to `low-1` holds purely `0` s.
  - `high+1` to `N` holds purely `2` s.
  - `mid` is our aggressive scanner. Everything between `mid` and `high` is unknown territory.
  - If `mid` hits a 0, we swap it to the `low` territory and advance both `low` and `mid`.
  - If `mid` hits a 1, it belongs in the middle anyway! Just advance `mid`.
  - If `mid` hits a 2, we swap it to the `high` territory and shrink `high`. **CRITICAL:** Do NOT advance `mid` here, because the element swapped back *from* `high` is unverified!
- **Optimal Java Code:**

```java
public class SortColors {
    public void sortColors(int[] nums) {
        int low = 0;
        int mid = 0;
        int high = nums.length - 1;

        while (mid <= high) {
            if (nums[mid] == 0) {
                swap(nums, low, mid);
                low++;
                mid++;
```

```
            } else if (nums[mid] == 1) {
                mid++;
            } else { // nums[mid] == 2
                swap(nums, mid, high);
                high--; // Notice: mid does NOT increment here
            }
        }
    }

    private void swap(int[] arr, int i, int j) {
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }
}
```

5. Dry Run

Input: `nums = [2, 0, 2, 1, 1, 0]`

- `low=0`, `mid=0`, `high=5`.

1. `mid(0) == 2`: Swap `mid` (0) & `high` (5). `nums=[0, 0, 2, 1, 1, 2]`. `high--` (4). (Notice `mid` stays `0`!)
2. `mid(0) == 0`: Swap `mid` (0) & `low` (0). `low++` (1), `mid++` (1).
3. `mid(1) == 0`: Swap `mid` (1) & `low` (1). `low++` (2), `mid++` (2).
4. `mid(2) == 2`: Swap `mid` (2) & `high` (4). `nums=[0, 0, 1, 1, 2, 2]`. `high--` (3). (`mid` stays `2`).
5. `mid(2) == 1`: Leave it. `mid++` (3).
6. `mid(3) == 1`: Leave it. `mid++` (4).
7. `mid(4) > high(3)`: Loop breaks. Sorted!

6. Architectural Thinking

- **WHAT:** Terrritorial Memory Partitioning.
- **WHERE:** Expanding solid safe-zones (`0` s on left, `2` s on right) while collapsing the chaotic unknown zone in the middle.
- **HOW:** Destructive in-place swapping driven purely by categorical `if/else` routing.
- **WHO:** This 3-way partition logic is exactly how the legendary "QuickSort" algorithm prevents `O(N^2)` worst-case explosions when arrays feature massive numbers of duplicates!

7. Edge Cases

- All 1s (`[1, 1, 1]`): `mid` races to the end, zero swaps execute. Optimal.
- Array already sorted (`[0, 1, 2]`): 0 swaps, 1 swap, loop ends.

🔹 8. Clean Code

- Abstracting the `.swap()` logic keeps the core loop elegantly readable and protects against accidental index overwrites.

🔹 9. Interview Training

- **Optimization Hint:** If the interviewer questions why you aren't incrementing `mid` when encountering a `2`, state boldly: "Because `high` represents unverified territory. Swapping an element from `high` to `mid` means my `mid` pointer is now looking at a completely unknown number. If I increment `mid` blindly, I permanently leave an unverified element in the middle zone, corrupting the array."

🔥 Problem 3: Best Time to Buy and Sell Stock

🔹 0. Company Tagging

- **Asked in:** Amazon, Google, Meta, Bloomberg
- **Frequency:** 🔥 High
- **Interview Context:** This is the foundational problem for greedy state accumulation. Finding the maximum difference between two elements where the smaller element must precede the larger element.

🔹 1. Problem Explanation

Given an array `prices` where `prices[i]` is the price of a stock on the `ith` day, maximize your profit by choosing exactly one day to buy and one future day to sell.

- **Real-world analogy:** Literal Stock Market day trading. You must buy before you can sell. You want the highest peak that occurs *after* the deepest valley.
- **Input:** `prices = [7, 1, 5, 3, 6, 4]`
- **Output:** `5` (Buy at 1 on Day 2, Sell at 6 on Day 5).
- **Constraints:** Cannot buy at 7 and sell at 1 for profit (time travels forward only).

🔹 2. Data Structure Masterclass

- **DS Used:** Array with primitive trackers.
- **Why chosen:** Pure sequential linear scan.

- **Time complexity benefits:** We don't need `O(N^2)` to compare every combination. `O(N)` allows real-time metric updates.

🔹 3. Pattern Recognition

- **Pattern name:** Greedy State Tracking (Min/Max synchronization).
- **WHEN to use:** When asked to find a maximum difference or maximum profit following a strict chronological order.
- **HOW to identify:** Keywords "buy and sell", "max profit", "one transaction".

🔹 4. Solution Evolution

✅ Approach 1: Brute Force

- **Thinking:** Use a double loop. For every day, check every subsequent day to see the profit margin. Track the maximum.
- **Complexity:** Time `O(N^2)`, Space `O(1)`. TLE on `10^5` constraints.

✅ Approach 2: Optimal Approach

- **Best solution idea:** I don't need to look ahead redundantly. As I scan day by day, what is my ultimate operational rule? **"To make maximum profit today, I should have bought the stock at the lowest price seen so far in history."**
  - So, use a variable `minPrice` to track the lowest historical price encountered.
  - On any day, `potentialProfit = currentPrice - minPrice`.
  - Update `maxProfit` if `potentialProfit` is higher.
- **Optimal Java Code:**

```java
public class StockBuySell {
    public int maxProfit(int[] prices) {
        int minPrice = Integer.MAX_VALUE; // Track lowest price seen so far
        int maxProfit = 0;                // Track absolute best profit
achievable

        for (int price : prices) {
            // Can we buy cheaper today?
            if (price < minPrice) {
                minPrice = price;
            }
            // If not buying today, let's see what happens if we sell today
            else {
                int currentProfit = price - minPrice;
                if (currentProfit > maxProfit) {
                    maxProfit = currentProfit;
                }
```

```
            }
        }

        return maxProfit;
    }
}
```

5. Dry Run

Input: `prices = [7, 1, 5, 3, 6, 4]`

- Init: `minPrice = INF`, `maxProfit = 0`

1. `p = 7`: `p < min` $=>$ `minPrice = 7`.
2. `p = 1`: `p < min` $=>$ `minPrice = 1`. (New absolute low!).
3. `p = 5`: `p > min`. `profit = 5 - 1 = 4`. `maxProfit = 4`.
4. `p = 3`: `p > min`. `profit = 3 - 1 = 2`. `maxProfit` stays 4.
5. `p = 6`: `p > min`. `profit = 6 - 1 = 5`. `maxProfit = 5`.
6. `p = 4`: `p > min`. `profit = 4 - 1 = 3`. `maxProfit` stays 5. Result: 5.

6. Architectural Thinking

- **WHAT:** Decoupling Future evaluation from Past dependencies via Caching.
- **WHERE:** A single chronological pass over time-series data.
- **HOW:** By persisting the optimal foundational state (`minPrice`) globally, every future node only needs an `O(1)` computation against cache, deleting the need for an inner loop!
- **WHEN:** Checked sequentially on every tick.
- **WHO:** This maps directly to High-Frequency Trading (HFT) stream processing algorithms that calculate deltas over rolling windows without buffering arrays!

7. Edge Cases

- **Descending Array (`[7, 6, 4, 3, 1]`):** `minPrice` updates constantly, but the `else` block NEVER executes because price never goes up. `maxProfit` remains `0`. You correctly lose no money. Correct behavior!

8. Clean Code

- **Readability:** Using pure `if/else` branching visually perfectly segregates "Buying Logic" from "Selling Logic". Avoid ternary operator bloat for this specific problem to keep the logic undeniably clear to reviewers.

9. Interview Training

- **Common follow-up:** "What if you can buy and sell multiple times?" (Stock Buy and Sell II).
  - Answer: "Then my strategy fundamentally flips from tracking global minimums to aggregating localized deltas. I would just add `prices[i] - prices[i-1]` to my absolute profit every time the price goes strictly up, turning it into a localized greedy accumulator!"

📝 DAILY SUMMARY

🎯 Patterns Learned Today

1. **Kadane's Aggressive Termination:** Resetting accumulators to absolute zeroes immediately when payload goes critically negative.
2. **Dijkstra's Dutch National Flag:** Swapping into solid territorial boundaries (`low` , `high`) while leaving unverified edge insertions dynamically in the testing loop (`mid`).
3. **Chronological Greedy State (Buy/Sell):** Resolving nested loop dependencies by maintaining a strictly historical `O(1)` marker (`min_history`) to instantaneously calculate `O(1)` operational deltas (`current - min_history`).

⚖️ Decision Rules

- If dealing with **Contiguous Subarray sums** featuring negatives $=>$ Use **Kadane's**.
- If dealing with **3 Distinct Element Categories** $=>$ Use **3-pointer DNF**.
- If doing **Maximum Delta / Profit** restricted chronologically $=>$ Use **Greedy Min Tracker**.

⚠ Key Mistakes to Avoid

- **Kadane's:** Returning `0` instead of `-1` on arrays with all negative numbers because you initialized `maxSum=0` . Always use `MIN_VALUE` .
- **Sort Colors:** Incrementing the `mid` pointer after swapping with `high` . If you swapped a `0` back from the end of the array to `mid` and skip it, your array will be permanently corrupted.

See you tomorrow for Day 3: Array Rotations & Merging!

Day 3: Arrays - Rotations & Merging

Welcome to Day 3! Today we tackle **Intervals and Dimensional Geometry**. These problems define the boundary between Junior developers (who use `O(N^2)` nested logic) and Senior developers (who use mathematical abstractions like Transposition and Gap calculations). Merging Intervals is heavily used in real-world system design for calendar scheduling and resource allocation.

🔥 Problem 1: Rotate Image (Matrix) by 90 Degrees

🔹 0. Company Tagging

- **Asked in:** Amazon, Microsoft, Apple
- **Frequency:** 🔥 High
- **Interview Context:** This problem tests your mathematical manipulation of 2D arrays. Allocating a second matrix to hold the rotated values is trivial. Manipulating the Cartesian coordinates *in-place* proves you understand memory addresses and linear algebra transpositions.

🔹 1. Problem Explanation

You are given an `n x n` 2D matrix representing an image. Rotate the image by **90 degrees clockwise**. You MUST rotate the image in-place, which means you have to modify the input 2D matrix directly.

- **Real-world analogy:** Rotating a photo on your iPhone. The software doesn't duplicate the 5MB photo in memory (which would crash low-RAM devices). It shifts the RGB pixel coordinates mathematically inside the existing RAM block.
- **Input:** `matrix = [[1,2,3],[4,5,6],[7,8,9]]`
- **Output:** `[[7,4,1],[8,5,2],[9,6,3]]`
- **Constraints:** Do NOT allocate another 2D matrix.

🔹 2. Data Structure Masterclass

- **DS Used:** 2D Array
- **Why chosen:** Required by problem for visual geometry representation.
- **Internal working:** Accessing `arr[i][j]` maps algebraically to $P_{row} + P_{col}$ memory offsets.

🔹 3. Pattern Recognition

- **Pattern name:** Mathematical Transposition + Reflection.
- **WHEN to use:** Any "Rotation" or "Mirroring" of a 2D grid.
- **HOW to identify:** Keywords "Rotate 90 degrees", "rotate matrix in-place".

🔹 4. Solution Evolution

✅ Approach 1: Brute Force

- **Thinking:** Create a dummy matrix `ans`. Map the elements: rule is that the *first row* of original becomes the *last column* of the rotated matrix. So, `ans[j][n - 1 - i] = matrix[i][j]`.
- **Complexity:** Time `O(N^2)`, Space `O(N^2)` extra memory. Fails the "in-place" requirement visually.

✅ Approach 2: Optimal Approach (Linear Algebra Transposition)

- **Best solution idea:** This is a famous Linear Algebra trick.
  - To rotate a matrix 90 degrees clockwise:
  - **Step 1:** Transpose the matrix (swap `[i][j]` with `[j][i]`). This flips the matrix diagonally.
  - **Step 2:** Reverse every row independently.
  - That's it!
- **Optimal Java Code:**

```java
public class RotateMatrix {
    public void rotateOptimal(int[][] matrix) {
        int n = matrix.length;

        // Step 1: Transpose the matrix
        // Iterate only the upper triangle (j > i) to prevent double-swapping!
        for (int i = 0; i < n; i++) {
            for (int j = i; j < n; j++) {
                int temp = matrix[i][j];
                matrix[i][j] = matrix[j][i];
                matrix[j][i] = temp;
            }
        }

        // Step 2: Reverse every row
        for (int i = 0; i < n; i++) {
            // Standard 1D array reversal using Left/Right pointers
            int left = 0;
            int right = n - 1;
            while (left < right) {
                int temp = matrix[i][left];
                matrix[i][left] = matrix[i][right];
                matrix[i][right] = temp;
                left++;
```

```
                right--;
            }
        }
    }
}
```

5. Dry Run

Input: `[[1,2,3], [4,5,6], [7,8,9]]`

- **Transpose Step:**
  - `i=0, j=1` : Swap 2 and 4. $=>$ Row 1 starts `[1, 4, 3]`, Row 2 starts `[2, 5, 6]`.
  - `i=0, j=2` : Swap 3 and 7.
  - `i=1, j=2` : Swap 6 and 8.
  - Transposed Matrix: `[[1,4,7], [2,5,8], [3,6,9]]`.
- **Reverse Rows Step:**
  - Row 0 (`[1,4,7]`): Swap 1 and 7. $=>$ `[7,4,1]`
  - Row 1 (`[2,5,8]`): Swap 2 and 8. $=>$ `[8,5,2]`
  - Row 2 (`[3,6,9]`): Swap 3 and 9. $=>$ `[9,6,3]`
- **Result:** `[[7,4,1], [8,5,2], [9,6,3]]`. Perfect 90 deg clockwise.

6. Architectural Thinking

- **WHAT:** Decoupling a complex geometric shift into two simple 1D mirror actions.
- **WHERE:** Executed sequentially via memory-pointer swapping.
- **HOW:** Transposing handles the column $->$ row transition. Reversing handles the direction vector (clockwise).
- **WHEN:** Checked sequentially, row by row.
- **WHO:** This abstraction is used everywhere in Game Development rendering engines to prevent allocating GPU memory for sprites!

7. Edge Cases

- **1x1 Matrix (`[[1]]`):** Both `for` loops safely skip or execute instantly without breaking.
- **Non-square Matrix (M x N)?** Rotating an `m x n` matrix in-place natively is impossible in hardware because the row/col memory chunk lengths physically change (A $2 x 3 `becomes` 3 x 2$, requiring a different physical memory block).

8. Clean Code

- Notice the transposition inner loop starts at `j = i`. If you start at `j = 0`, you will swap the elements, and then when the loop reaches the mirrored element later, you will swap them BACK to their original positions.

🔹 9. Interview Training

- **Mistakes to avoid:** Do not try to trace individual coordinates circularly (e.g., `temp = top_left; top_left = bottom_left...`). While technically `O(1)` space and `O(N^2)` time, the coordinate logic mapping is a nightmare to code bug-free on a whiteboard. Transpose + Reverse is mathematically elegant and physically impossible to mess up if you know how to reverse a 1D array.

🔥 Problem 2: Merge Overlapping Intervals

🔹 0. Company Tagging

- **Asked in:** Google, Amazon, Meta, Microsoft
- **Frequency:** 🔥 High
- **Interview Context:** This is the quintessential calendar scheduling problem. It tests if you understand how Sorting groups relational concepts physically together, enabling `O(N)` linear evaluations instead of `O(N^2)` relation checking.

🔹 1. Problem Explanation

Given an array of `intervals` where `intervals[i] = [start_i, end_i]`, merge all overlapping intervals, and return an array of the non-overlapping intervals that cover all the intervals in the input.

- **Real-world analogy:** Merging overlapping calendar meetings. If Meeting A is 1:00-3:00, and Meeting B is 2:00-4:00, your total "Busy block" is 1:00-4:00.
- **Input:** `intervals = [[1,3],[2,6],[8,10],[15,18]]`
- **Output:** `[[1,6],[8,10],[15,18]]` (Since 1-3 and 2-6 overlap, they merged into 1-6).

🔹 2. Data Structure Masterclass

- **DS Used:** `List<int[]>`
- **Why chosen:** The output size is dynamic. We don't know how many intervals will merge. An ArrayList is necessary to buffer the output.
- **Time complexity benefits:** Sorting the array takes `O(N log N)`. Once sorted, overlapping intervals are guaranteed to sit adjacent to each other physically in the array!

🔹 3. Pattern Recognition

- **Pattern name:** Sorted Interval Merging.
- **WHEN to use:** Questions asking about chronological overlaps, meeting rooms, or range intersections.
- **HOW to identify:** Keywords "overlapping", "intervals".

🔹 4. Solution Evolution

✅ Approach 1: Brute Force

- **Thinking:** Check every interval against every other interval. If they overlap, create a new interval and delete the old ones.
- **Complexity:** Time `O(N^2)`, Space `O(1)`. Tricky to implement, deletions cause array shifting.

✅ Approach 2: Optimal Approach (Sort + Line Sweep)

- **Best solution idea:** The chaotic problem is that `[1,3]` could be at index 0, and `[2,4]` could be at index 1000.
  - If we SORT the intervals based on their **Start Times**, overlapping intervals are forced to be next to each other.
  - We pick up the first interval. We look at the next interval.
  - If `Next.start <= Current.end`, they OVERLAP! The new interval is `[Current.start, MAX(Current.end, Next.end)]`.
  - If `Next.start > Current.end`, they DON'T overlap. We push the `Current` interval to the Answer List, and "pick up" the `Next` interval to start checking.
- **Optimal Java Code:**

```java
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class MergeIntervals {
    public int[][] mergeOptimal(int[][] intervals) {
        if (intervals.length <= 1) return intervals;

        // 1. Sort the intervals based on START TIME
        // Using an anonymous Comparator lambda
        Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));

        List<int[]> merged = new ArrayList<>();
```

```java
        // Pick up the very first interval to act as our baseline tracker
        int[] currentInterval = intervals[0];
        merged.add(currentInterval); // Add reference to list (we will mutate
its End Time later!)

        for (int[] nextInterval : intervals) {
            int currentEnd = currentInterval[1];
            int nextStart = nextInterval[0];
            int nextEnd = nextInterval[1];

            if (nextStart <= currentEnd) {
                // Overlap! Mutate the End Time of the interval currently
sitting in the List
                currentInterval[1] = Math.max(currentEnd, nextEnd);
            } else {
                // No Overlap! Pick up the next interval and add it to the list
                currentInterval = nextInterval;
                merged.add(currentInterval);
            }
        }

        // Convert List<int[]> back to a primitive 2D matrix
        return merged.toArray(new int[merged.size()][]);
    }
}
```

5. Dry Run

Input: `[[1,3], [2,6], [8,10], [15,18]]`. (Already sorted!).

- Load `[1, 3]` into list. `currentInterval` tracks `[1, 3]`.
- Loop 1 (`[1,3]`): `1 <= 3` (Overlap). End `max(3, 3)` = 3. (`[1,3]` remains `[1,3]`).
- Loop 2 (`[2,6]`): `2 <= 3` (Overlap!). Mutate End `max(3, 6)` = 6. List now holds `[[1,6]]`.
- Loop 3 (`[8,10]`): `8 > 6` (No Overlap). New distinct block! Add `[8,10]` to List. `currentInterval` tracks `[8,10]`.
- Loop 4 (`[15,18]`): `15 > 10` (No Overlap). Add `[15,18]`. `currentInterval` tracks `[15,18]`.
- Return `[[1,6],[8,10],[15,18]]`.

🔹 6. Architectural Thinking

- **WHAT:** Sequential Line-Sweeping via physical grouping.
- **WHERE:** Tracking state using an independent reference pointer mutated dynamically.
- **HOW:** Sorting enforces chronological monotonicity. After sorting, timeline breaks are strictly disjoint!
- **WHEN:** The reference interval acts as an accumulator until a chronological anomaly (gap) severs the accumulation.
- **WHO:** Sorting is the Ultimate "Pre-computation" mechanism in System Design databases to group similar metadata nodes together.

🔹 7. Edge Cases

- **Complete engulfing (`[[1, 10], [2, 3]]`):** The `Math.max(10, 3)` cleanly keeps `10`, merging the smaller interval seamlessly.
- **Negative timestamps?** Handled natively by `<` comparisons.

🔹 8. Clean Code

- **Pro-tip:** `merged.add(currentInterval)` adds the *memory reference* to the list. When we do `currentInterval[1] = ...`, it mutates the array *inside* the List instantly without needing a `.set()` or removal hook! This is extremely elegant Java pointer manipulation.

🔹 9. Interview Training

- **Junior Move:** Writing a custom Merge Sort or Quick Sort from scratch to sort the 2D array.
- **Senior Move:** using `Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]))`. This lambda implicitly creates a highly optimized Comparator. Never write a sorting loop in production unless explicitly asked "implement a sorting function."

🔥 Problem 3: Merge Two Sorted Arrays Without Extra Space

🔹 0. Company Tagging

- **Asked in:** Google, Amazon, Microsoft
- **Frequency:** 🔥 High
- **Interview Context:** This is an edge-case test on spatial manipulation. If you use a third array to merge `O(M+N)` space, you fail. The problem is testing if you can conceptually "Shift" numbers backwards, or use advanced mathematical gap logic (Shell sort).

🔹 1. Problem Explanation

Given two sorted integer arrays `nums1` and `nums2`, merge `nums2` into `nums1` as one sorted array. The number of elements initialized in `nums1` and `nums2` are `m` and `n` respectively. `nums1` has a size equal to `m + n` such that it has enough space to hold all elements from `nums2`.

- **Real-world analogy:** You have two sorted filing cabinets. Cabinet A has empty space in the back. You need to pull files from B and weave them into A without placing any files on the floor (no extra memory buffer).
- **Constraints:** Maximize time efficiency, `O(1)` space.

🔹 2. Data Structure Masterclass

- **DS Used:** Two 1D Primitive Arrays.
- **Why chosen:** Fixed block manipulation.
- **Time complexity benefits:** Modifying arrays backwards avoids `O(N)` index-shifting penalties.

🔹 3. Pattern Recognition

- **Pattern name:** Three-Pointer Backward Merging.
- **WHEN to use:** "Merge sorted lists inside an existing buffer" or "Avoid index shifting".

🔹 4. Solution Evolution

✅ Approach 1: Brute Force (Better than standard Brute)

- **Thinking:** Put all elements of `nums2` into the empty slots of `nums1`. Then run `Arrays.sort(nums1)`.
- **Complexity:** Time `O((M+N) log(M+N))`, Space `O(1)` (ignoring native sorting heap). This is technically `O(1)` space, but the time is heavily suboptimal.

✅ Approach 2: Optimal Approach (Backward Pointers)

- **Best solution idea:** If we iterate from the *front* (`index 0`), if `nums2` has a small element, we have to shove it into `nums1[0]` and then shift EVERY other element in `nums1` to the right, taking `O(M x N)` time.
  - BUT... the back of `nums1` is completely empty (filled with `0` s)!
  - If we compare the *largest* element of `nums1` vs the *largest* element of `nums2`, we know EXACTLY which one belongs at the very end of the result array (`nums1[m+n-1]`).

- By writing data from the back to the front, we never overwrite data we haven't processed yet!

- **Optimal Java Code:**

```java
public class MergeSortedArrays {
    public void mergeOptimal(int[] nums1, int m, int[] nums2, int n) {
        // Pointers for the valid data sections
        int p1 = m - 1;
        int p2 = n - 1;
        // Pointer for writing into the padded empty zeros at the rear of nums1
        int pMerge = m + n - 1;

        // While there is data in BOTH trackers
        while (p1 >= 0 && p2 >= 0) {
            // Pick the strictly LARGEST element and place it in the rear
            if (nums1[p1] > nums2[p2]) {
                nums1[pMerge] = nums1[p1];
                p1--;
            } else {
                nums1[pMerge] = nums2[p2];
                p2--;
            }
            pMerge--;
        }

        // If nums1 is exhausted, but nums2 still has small elements remaining,
        // we must copy them over. (If nums2 is exhausted, nums1's remaining
elements
        // are already in their correct sorted positions!)
        while (p2 >= 0) {
            nums1[pMerge] = nums2[p2];
            p2--;
            pMerge--;
        }
    }
}
```

5. Dry Run

Input: `nums1 = [1, 2, 3, 0, 0, 0]`, `m = 3`. `nums2 = [2, 5, 6]`, `n = 3`.

- `p1 = 2` (val=3). `p2 = 2` (val=6). `pMerge = 5`.

- Compare 3 & 6. 6 is bigger. `nums1[5] = 6`. `p2--` (now points to 5). `pMerge--` (now 4).
- `p1(3)` vs `p2(5)`. 5 is bigger. `nums1[4] = 5`. `p2--`. `pMerge--`.
- `p1(3)` vs `p2(2)`. 3 is bigger. `nums1[3] = 3`. `p1--`. `pMerge--`.
- `p1(2)` vs `p2(2)`. Equal. `else` block places `nums2[2]`. `nums1[2] = 2`. `p2--`.
- `p2` is now -1. Exhausted!
- `nums2` loop skips. Result: `[1, 2, 2, 3, 5, 6]`. Done!

🔹 6. Architectural Thinking

- **WHAT:** Reverse Traversal Overlap Prevention.
- **WHERE:** Operating from the highest-indexed unallocated buffer downward.
- **HOW:** The memory layout guarantees the unused buffer is large enough to absorb all incoming data without colliding with the read-pointer `p1`.
- **WHEN:** Synchronized comparison loops.
- **WHO:** Moving elements backwards is a fundamental low-level optimization in Assembly Language to execute `memcpy` on overlapping blocks!

🔹 7. Edge Cases

- **`nums1` is empty initially (`m=0`):** `p1 = -1`. First `while` loop completely skips. Second `while` loop executes wildly, copying all of `nums2` into `nums1`. Perfect.
- **`nums2` is empty (`n=0`):** `p2 = -1`. Second `while` skips. `nums1` is untouched. Perfect.

🔹 8. Clean Code

- Adding `0` padding is universally how empty buffer space is allocated in C/Java statically. Trusting the bounds variables `m` and `n` instead of `.length` ensures we only map actual data payloads.

🔹 9. Interview Training

- **The Junior Pitfall:** The junior thinks: "Shift the array elements right using a `for` loop to make space for `nums2[i]`". That runs in `O(M^2)`.
- **The Senior Insight:** Write from the back! Always remember: if an array has padded zeros at the back, it's begging you to iterate backward.

📝 DAILY SUMMARY

🎯 Patterns Learned Today

1. **Mathematical Spatial Mapping:** Using native array Transpose and Reverse procedures to physically rotate states `O(N)` without allocating new geometry boards (`Rotate Image`).
2. **Lambda Pre-sorting Monotonicity:** Applying Custom `Comparators` onto complex multidimensional types to force chronological adjacency `[Merge Intervals]` .
3. **Backward Pointer Collision Avoidance:** Exploiting "Trailing Zeros" space to safely merge distinct arrays asynchronously into native buffer slots `[Merge Sorted Arrays]` .

⚖️ Decision Rules

- If manipulating **2D Matrix Coordinates** $=>$ Look for mathematical transposition rules. Do not manually swap complex corner pointers.
- If dealing with **Intervals or Ranges** $=>$ ALWAYS sort by the `Start Time` . Monotonic sorting trivially collapses intersections via simple `max()` End Time tracking.
- If asked to **Merge Arrays implicitly** without allocating new ones $=>$ Run pointers backwards.

⚠ Key Mistakes to Avoid

- **Rotate Image:** Initializing the inner `j` loop of the Transpose equation to `0` . It must start at `i` (the diagonal), otherwise, you `[i][j] = [j][i]` and then immediately reverse it later on `[j][i] = [i][j]` , resulting in no change!
- **Merge Intervals:** Forgetting to mutate the `int[]` interval *reference* sitting inside the ArrayList to mathematically expand the final bounding box.

See you tomorrow for Day 4: Hashing & Two Pointers Fundamentals!

Day 4: Hashing & Two Pointers Fundamentals

Welcome to Day 4! Today marks a paradigm shift. So far, we have manipulated data exclusively within contiguous memory constraints (Arrays). Today, we introduce **Hashing**—sacrificing predictable `O(N)` Space to achieve instantaneous `O(1)` Time lookups. Combining Hashing with strict sequential logic (Two Pointers) creates the most ubiquitous problem-solving framework in all of Software Engineering.

🔥 Problem 1: Two Sum

🔹 0. Company Tagging

- **Asked in:** Amazon, Google, Apple, Meta
- **Frequency:** 🔥 High
- **Interview Context:** This is arguably the most famous interview question on earth (LeetCode #1). It tests a very specific conceptual leap: Can you transition from a "Look-Ahead" mentality (`O(N^2)` brute force) to a "Look-Behind" mentality using structural caching?

🔹 1. Problem Explanation

Given an array of integers `nums` and an integer `target`, return indices of the two numbers such that they add up to `target`. You may assume exactly one solution exists, and you may not use the same element twice.

- **Real-world analogy:** You have $100. You need to buy exactly two items from a store that equal exactly $100. You pick up a $40 shirt. Instead of searching the whole store again for a $60 item, you ask the cashier: "Hey, have you already seen a $60 item today?"
- **Input:** `nums = [2, 7, 11, 15]`, `target = 9`
- **Output:** `[0, 1]`
- **Constraints:** Must be slower than `O(N^2)`, optimally `O(N)`.

🔹 2. Data Structure Masterclass

- **DS Used:** `HashMap<Integer, Integer>`
- **Why chosen:** Arrays only allow `O(1)` retrieval if you know the *Index*. HashMaps allow `O(1)` retrieval using the *Value itself* as the Key!
- **Internal working:** The integer Key is passed through Java's `.hashCode()` formula to generate a bucket index. The Value (the array index) is stored there.
- **Trade-offs:** We allocate `O(N)` extra memory on the Heap. Hardware caches (L1/L2) hate HashMaps compared to contiguous Arrays due to pointer indirection.

🔹 3. Pattern Recognition

- **Pattern name:** Algebraic Complement Hashing (Look-Behind Cache).
- **WHEN to use:** When searching for independent pairs that satisfy a mathematical formula (`A + B = Target`) in an **UNSORTED** array.
- **HOW to identify:** Keywords "Two elements", "Sum to target", "Return indices".

🔹 4. Solution Evolution

✅ Approach 1: Brute Force (Look-Ahead)

- **Thinking:** For every number `A` , scan the rest of the array looking for `Target - A` .
- **Code:**

```java
public int[] twoSumBruteForce(int[] nums, int target) {
    for (int i = 0; i < nums.length; i++) {
        for (int j = i + 1; j < nums.length; j++) {
            if (nums[i] + nums[j] == target) {
                return new int[]{i, j};
            }
        }
    }
    return new int[]{};
}
```

- **Complexity:** Time `O(N^2)` , Space `O(1)` .

✅ Approach 2: Better Approach (Sorting + Two Pointers)

- **Idea:** What if we sort the array? Then we can put a pointer at `left=0` and `right=N-1` and squeeze inward!
- **Fatal Flaw:** The problem asks to return the *original indices*. Sorting the array scrambles the original indices! You would have to create a custom `Node` class containing `{value, originalIndex}` , sort an array of Nodes, and then Two-Pointer it.
- **Complexity:** Time `O(N log N)` , Space `O(N)` (for the Node array). If we are using `O(N)` space anyway, we can do faster than `O(N log N)` !

✅ Approach 3: Optimal Approach (One-Pass HashMap)

- **Best solution idea:** Create an algebraic equation. `CurrentValue + X = Target` mathematically means `X = Target - CurrentValue` .
    - As we iterate through the array, we check: "Is `X` in my HashMap of past numbers?"

- If YES: We found it! We return `[Map.get(X), CurrentIndex]`.
- If NO: We store `[CurrentValue, CurrentIndex]` into the Map for future numbers to find!

- **Optimal Java Code:**

```java
import java.util.HashMap;

public class TwoSum {
    public int[] twoSumOptimal(int[] nums, int target) {
        // Map stores (Number Value -> Original Index)
        HashMap<Integer, Integer> map = new HashMap<>();

        for (int i = 0; i < nums.length; i++) {
            int currentNum = nums[i];
            int requiredComplement = target - currentNum;

            // Look-Behind: Did we already see the required number?
            if (map.containsKey(requiredComplement)) {
                return new int[]{map.get(requiredComplement), i};
            }

            // Record current state for the future
            map.put(currentNum, i);
        }

        throw new IllegalArgumentException("No two sum solution");
    }
}
```

5. Dry Run

Input: `nums = [3, 2, 4]`, `target = 6`

1. `i = 0` (val `3`). `complement = 6 - 3 = 3`. Map has 3? No. Map.put(`3 $->$ 0`).
2. `i = 1` (val `2`). `complement = 6 - 2 = 4`. Map has 4? No. Map.put(`2 $->$ 1`).
3. `i = 2` (val `4`). `complement = 6 - 4 = 2`. Map has 2? YES! Value is index `1`.
4. Return `[1, 2]`.

6. Architectural Thinking

- **WHAT:** `O(1)` Historical state querying via Hash Indexing.
- **WHERE:** Stream processing data lazily.
- **HOW:** Storing the inverse-computational requirement dynamically. We don't store "what we need", we store "what we have", and query "what we need" against it.
- **WHEN:** Checked synchronously *before* the current state is committed to memory to prevent a number from matching against itself!
- **WHO:** This pattern mimics Session Caching mechanisms in stateless web servers!

🔹 7. Edge Cases

- **Duplicate Elements (`[3, 3]` , target 6):** The order of operations is brilliant here. At `i=0` (val 3), map is empty, we put `3->0` . At `i=1` (val 3), complement `3` is IN the map! It returns `[0, 1]` without ever overwriting the map with `3->1` .
- **Negatives (`[-1, -2, 3]` , target 1):** Math `-1 + 2 = 1` natively. Java handles signed integers automatically.

🔹 8. Clean Code

- **Throwing Exceptions:** In production, if a method guarantees a return array but the loop finishes without finding one, throwing an `IllegalArgumentException` is vastly superior to returning `null` or `new int[]{}` which causes silent NullPointerExceptions upstream.

🔹 9. Interview Training

- **Senior Insight:** Interviewers might ask: "Is the HashMap ALWAYS better?"
  - Answer: "Actually, no. If the array is massive and memory is highly constrained (embedded systems), allocating `O(N)` Heap space for a Map might trigger an OutOfMemoryError. In that scenario, the Sorting + Two Pointers approach (`O(N log N)` Time, `O(1)` Space) is architecturally safer despite being mathematically slower. Best tool for the specific constraint."

🔥 Problem 2: 4Sum

🔹 0. Company Tagging

- **Asked in:** Amazon, Apple, Meta
- **Frequency:** ⚡ Medium
- **Interview Context:** Anyone can solve Two Sum. This problem tests your ability to abstract higher-order complexity (K-Sum) down into fundamental primitives. It strictly tests your mastery of avoiding duplicates without using excessive HashSets.

🔹 1. Problem Explanation

Given an array `nums` of `n` integers, return an array of all the **unique** quadruplets `[nums[a], nums[b], nums[c], nums[d]]` such that they sum to `target`.

- **Real-world analogy:** Combining exactly 4 specific ingredients from a pantry to make a recipe weighing exactly 10 lbs. You don't want to list "Flour, Sugar, Eggs, Milk" and then again "Eggs, Flour, Milk, Sugar" as two different recipes.
- **Constraints:** `a, b, c, d` are distinct indices. Output must NOT contain duplicate quadruplets.

🔹 2. Data Structure Masterclass

- **DS Used:** `List<List<Integer>>` + Sorted Array.
- **Why chosen:** Sorting intrinsically groups duplicates ($1, 4, 4, 4, 7$). This allows us to use simple `if (nums[i] == nums[i-1]) skip` logic instead of buffering millions of heavy `String` hashes into a Set.

🔹 3. Pattern Recognition

- **Pattern name:** K-Sum Reduction (Anchor loops + Two Pointers).
- **WHEN to use:** Asking for 3, 4, or K elements summing to a target, demanding Unique Sets.
- **HOW to identify:** "Unique quadruplets", "4Sum", "Target".

🔹 4. Solution Evolution

✅ Approach 1: Brute Force

- **Thinking:** 4 nested loops. Sum everything. To ensure randomness is killed, sort every matching quadruplet and add to a `HashSet`.
- **Complexity:** Time `O(N^4)`, Space `O(K)` for HashSet. Awful.

✅ Approach 2: Better Approach (3 Loops + Binary Search/HashSet)

- **Thinking:** Fix 3 elements with loops. Use Binary Search (`O(log N)`) or a HashMap to find the 4th element.
- **Complexity:** Time `O(N^3 log N)`, Space `O(N)`. Still very slow.

✅ Approach 3: Optimal Approach (Sort + Two Pointers)

- **Best solution idea:** The 2Sum problem took 2 pointers (L, R). The 3Sum problem took 1 anchor loop `i`, plus 2 pointers (L, R). Following the fractal pattern, 4Sum takes 2 anchor loops (`i` , `j`), plus 2 pointers (L, R)!
  - Step 1: Sort the array.
  - Step 2: Loop `i` from $0 -> N-3$. (Skip duplicates).

- Step 3: Loop `j` from $i+1 -> N-2$. (Skip duplicates).
- Step 4: The problem is now reduced to 2Sum! Target is `target - (nums[i] + nums[j])`. Squeeze `L` and `R` inward.

- **Optimal Java Code:**

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class FourSum {
    public List<List<Integer>> fourSumOptimal(int[] nums, int target) {
        List<List<Integer>> result = new ArrayList<>();
        int n = nums.length;
        if (n < 4) return result;

        // 1. Sort the array to group duplicates and enable L/R convergence
        Arrays.sort(nums);

        for (int i = 0; i < n - 3; i++) {
            // Skip duplicate anchor i
            if (i > 0 && nums[i] == nums[i - 1]) continue;

            for (int j = i + 1; j < n - 2; j++) {
                // Skip duplicate anchor j
                if (j > i + 1 && nums[j] == nums[j - 1]) continue;

                int left = j + 1;
                int right = n - 1;

                while (left < right) {
                    // Use long to prevent integer overflow adding 4 large
numbers
                    long sum = (long)nums[i] + nums[j] + nums[left] +
nums[right];

                    if (sum == target) {
                        result.add(Arrays.asList(nums[i], nums[j], nums[left],
nums[right]));

                        // Shrink pointers
                        left++;
                        right--;

                        // Skip duplicates for left and right
                        while (left < right && nums[left] == nums[left - 1])
```

```
left++;
                        while (left < right && nums[right] == nums[right + 1])
right--;

                    } else if (sum < target) {
                        left++;
                    } else {
                        right--;
                    }
                }
            }
        }
        return result;
    }
}
```

5. Dry Run

Input: `nums = [1, 0, -1, 0, -2, 2]`, `target = 0`

- Sort: `[-2, -1, 0, 0, 1, 2]`
- `i=0` (-2). `j=1` (-1).
  - `L=2` (0), `R=5` (2). Sum: `-2 -1 + 0 + 2 = -1 < 0`. `L++`.
  - `L=3` (0), `R=5` (2). Sum: `-2 -1 + 0 + 2 = -1 < 0`. `L++`.
  - `L=4` (1), `R=5` (2). Sum: `-2 -1 + 1 + 2 = 0`. MATCH! Add `[-2, -1, 1, 2]`.
- `i=0` (-2). `j=2` (0).
  - `L=3` (0), `R=5` (2). Sum: `-2 + 0 + 0 + 2 = 0`. MATCH! Add `[-2, 0, 0, 2]`.
- (The loops natively leap over the duplicate `0`s due to the `continue` and inner `while` checks, producing strictly unique answers!)

6. Architectural Thinking

- **WHAT:** Dimensionality Reduction via loop fixing.
- **WHERE:** Operating recursively on sub-arrays (`j+1 -> N-1`).
- **HOW:** Iteratively locking higher-dimensional variables to collapse an `O(N^4)` problem space strictly down into the foundational `O(N)` 2Sum paradigm.
- **WHEN:** Duplicate stripping is handled synchronously as pointers traverse the data.
- **WHO:** This abstraction is the architectural core of compiling relational SQL JOIN queries!

🔹 7. Edge Cases

- **Integer Overflows:** The test cases famously feature Arrays with `Integer.MAX_VALUE`. Adding them up overflows instantly. Casting to `(long)` during the `sum` calculation is a strictly Senior-level requirement.

🔹 8. Clean Code

- **Readability:** Keep the 4 inner skipping mechanisms visually distinct.
- `if (i > 0 && nums[i] == nums[i - 1])`
- `if (j > i + 1 && nums[j] == nums[j - 1])`
- `while (left < right && nums[left] == nums[left - 1])`
- `while (left < right && nums[right] == nums[right + 1])`

🔹 9. Interview Training

- **Mistakes to avoid:** Using a `HashSet<List<Integer>>` to filter duplicates at the return statement. Interviewers view this as lazy and inefficient. Mastering the adjacent duplicate skipping logic (`if a == a-1 continue`) proves raw algorithmic mastery.

🔥 Problem 3: Longest Consecutive Sequence

🔹 0. Company Tagging

- **Asked in:** Google, Amazon, Microsoft
- **Frequency:** 🔥 High
- **Interview Context:** This problem seems impossible on paper. "Find the longest sequence in `O(N)` time in an UN-SORTED array". It tests if you understand how to utilize HashSets not just for searching, but as isolated Graph nodes.

🔹 1. Problem Explanation

Given an unsorted array of integers `nums`, return the length of the longest consecutive elements sequence.

- **Real-world analogy:** You have a scattered pile of puzzle pieces with numbers on them. You want to snap together the longest connected chain (`100, 101, 102`). Finding them one by one in the pile takes forever.
- **Input:** `nums = [100, 4, 200, 1, 3, 2]`
- **Output:** `4` (The sequence is `[1, 2, 3, 4]`).
- **Constraints:** Must execute strictly in `O(N)` time!

🔹 2. Data Structure Masterclass

- **DS Used:** `HashSet<Integer>`
- **Why chosen:** Lookups take exactly `O(1)` time.
- **Time complexity benefits:** We can theoretically check if `current_number + 1` exists in the array instantly without scanning the array!

🔹 3. Pattern Recognition

- **Pattern name:** Intelligent Sequence Building (Disjoint Set abstraction).
- **WHEN to use:** "Consecutive Sequence", "Unsorted array", "O(N) time constraint".

🔹 4. Solution Evolution

✅ Approach 1: Brute Force

- **Thinking:** For every number, linearly check if `num+1` exists in the array, then `num+2`, etc.
- **Complexity:** Time `O(N^3)` worst case. Disaster.

✅ Approach 2: Better Approach (Sorting)

- **Thinking:** Just sort the array! Then run one loop counting how many adjacent elements increase by exactly `1`.
- **Complexity:** Time `O(N log N)` (violates `O(N)` rule). Space `O(1)`. It's actually a very decent fallback if you forget the optimal approach in an interview, but won't get you a "Strong Hire" rating at Google.

✅ Approach 3: Optimal Approach (HashSet Sequence Builder)

- **Best solution idea:**
  - Dump all numbers into a `HashSet`. This gives us `O(1)` lookups.
  - Iterate over the Set. Pick a number (like `100`).
  - **The Genius Move:** *Is `100` the START of a sequence?* How do we know? If `99` exists in the set, then `100` is NOT the start! It's just a middle piece! We skip it instantly.
  - If `99` does NOT exist in the set, then `100` IS the start of a sequence! From there, we use a `while(set.contains(num + 1))` loop to count upwards (`101`, `102`...).
- **Why does this beat `O(N^2)`?** The inner `while` loop ONLY triggers if the number is the absolute start of a sequence. This guarantees every number is visited exactly twice overall (once building the map, once evaluated in the inner loop). `O(2N) = O(N)`.
- **Optimal Java Code:**

```java
import java.util.HashSet;

public class LongestConsecutive {
    public int longestConsecutiveOptimal(int[] nums) {
        if (nums == null || nums.length == 0) return 0;

        HashSet<Integer> set = new HashSet<>();
        for (int num : nums) {
            set.add(num);
        }

        int longestStreak = 0;

        for (int num : set) { // Iterate over set to avoid duplicate start-points
            // Look-Behind: Is this number the true start of a sequence?
            // If num - 1 exists, this 'num' is just a middle piece. Skip it!
            if (!set.contains(num - 1)) {

                int currentNum = num;
                int currentStreak = 1;

                // Look-Ahead: Count upwards as long as the chain connects
                while (set.contains(currentNum + 1)) {
                    currentNum += 1;
                    currentStreak += 1;
                }

                longestStreak = Math.max(longestStreak, currentStreak);
            }
        }

        return longestStreak;
    }
}
```

5. Dry Run

Input: `nums = [100, 4, 200, 1, 3, 2]`

- Map state: `{100, 4, 200, 1, 3, 2}`
- Loop `num=100`: Concept check: Does 99 exist? No! Sequence bounds `[100]`. Loop checks 101. Doesn't exist. Max=1.
- Loop `num=4`: Does 3 exist? YES! It's not a start. Skip!

- Loop `num=200` : Does 199 exist? No! Bounds `[200]` . 201 missing. Max=1.
- Loop `num=1` : Does 0 exist? No! It's a start!
  - `while(contains(1 + 1))` : found 2. Streak=2.
  - `while(contains(2 + 1))` : found 3. Streak=3.
  - `while(contains(3 + 1))` : found 4. Streak=4.
  - `while(contains(4 + 1))` : not found. End. Max=4.
- Loop `num=3` : Does 2 exist? Yes! Skip.
- Result: 4.

6. Architectural Thinking

- **WHAT:** Graph theory Connected Components via Dictionary abstractions.
- **WHERE:** Uncoupled nodes floating in a Hash table.
- **HOW:** Exploiting the `!contains(n-1)` check as a theoretical "Root Node" finder for disconnected graph chains. By only traversing starting from root nodes, we prevent redundant loop cycles!
- **WHEN:** Stream evaluation mapped directly to hash space.
- **WHO:** Using Sets to natively group continuous memory segments is exactly how Garbage Collectors evaluate contiguous Heap blocks!

7. Edge Cases

- **Empty Array:** Triggers `length == 0` guard clause. Returns 0.
- **Duplicate Sequences (`[1, 2, 0, 1]`):** The `HashSet` ingests duplicates silently (`{1, 2, 0}`). It processes `0` , finds `1` and `2` , yielding length 3 seamlessly.

8. Clean Code

- **Readability:** Iterating `for (int num : set)` instead of `for (int num : nums)` is a massive micro-optimization. If the input is `[0, 0, 0, 0]` , iterating the set triggers the outer loop exactly once. Iterating `nums` triggers it four times needlessly.

9. Interview Training

- **Senior Check:** The interviewer WILL ask: "*You have a `for` loop, and an inner `while` loop. Are you SURE this is `O(N)` time?*"
- **The Golden Answer:** "*Yes. The `if (!set.contains(num - 1))` is a mathematical gatekeeper. It guarantees the inner `while` loop ONLY executes for the very first element of any sequence. Therefore, across the entire duration of the program, the `while` loop will only step*

over each array element exactly once. Over `N` iterations, the inner loop fires cumulatively `N` times, making it strictly `O(N + N) = O(N)`."

📝 DAILY SUMMARY

🎯 Patterns Learned Today

1. **Look-Behind Caching (Two Sum):** Flipping calculations backwards to turn `O(N^2)` look-aheads into `O(1)` historical Hash queries.
2. **K-Sum Dimensional Collisions (4Sum):** Utilizing monotonic sorting and skip-logic to compress 4 degrees of freedom into 2 anchored loops and a linear squeeze.
3. **Graph Root Identification (Longest Sequence):** Applying rule-based bounds `!contains(n-1)` to logically identify the head of a sequence amidst pure random distribution, circumventing Sorting matrices entirely.

⚖️ Decision Rules

- If given a math problem and **Target K** in an **unsorted** array $=>$ Think `HashMap` `(Current + X = K)`.
- If requested to provide combinations of **Unique Sub-elements** $=>$ ALWAYS sort the array, and use strict indexing skips (`a == a-1`). HashSets for duplicate removal are inefficient crutches.
- If finding the **Longest Sequence** in an `O(N)` constraint $=>$ Dump to a HashSet, and logically hunt exclusively for the "Start Nodes" of the sequences.

⚠ Key Mistakes to Avoid

- **Two Sum:** Using the same element twice. Always check the hashmap *before* putting the current element into the hashmap!
- **4Sum:** Overlooking the Integer Overflow bug when adding 4 array elements together. Use `(long)` carefully!
- **Longest Consecutive Sequence:** Forgetting the `!contains(n-1)` check. Without it, the solution deteriorates back into a catastrophic `O(N^2)` timeframe.

See you tomorrow for Day 5: Hashing & Prefix Sum Advanced!

Day 5: Hashing & Prefix Sum Advanced

Welcome to Day 5! If Day 4 taught us how to query single elements in `O(1)` time, Day 5 teaches us how to query **entire sub-ranges of history** in `O(1)` time. This is where we master the `Prefix Array` logic combined with HashMaps. This logic is arguably the most requested array pattern for Senior-level Backend roles, as it perfectly mimics rolling log aggregations and distributed event stream analysis.

🔥 Problem 1: Largest Subarray with 0 Sum

🔹 0. Company Tagging

- **Asked in:** Amazon, Microsoft, ByteDance
- **Frequency:** 🔥 High
- **Interview Context:** This tests your transition from "Two Pointers" to "Prefix Sums". Standard Two Pointers only works if the array is Monotonic (strictly positive integers). Once negative numbers are introduced, sliding windows break. You *must* recognize that algebraic equations on running totals are required.

🔹 1. Problem Explanation

Given an array containing both positive and negative integers, find the length of the longest subarray with a sum equal to 0.

- **Real-world analogy:** Finding the longest period where a company's financial profit and loss exactly canceled each other out to dead-even 0.
- **Input:** `nums = [15, -2, 2, -8, 1, 7, 10, 23]`
- **Output:** `5` (The subarray `[-2, 2, -8, 1, 7]` sums exactly to 0.)
- **Constraints:** Maximize time efficiency (`O(N)` expected).

🔹 2. Data Structure Masterclass

- **DS Used:** `HashMap<Integer, Integer>` (Prefix Sum Map).
- **Why chosen:** We need to store mathematical states (the running sum) as the Key, and geographical points (the original index) as the Value.
- **Time complexity benefits:** Re-calculating `sum(i, j)` natively takes `O(N)` time. By hashing `PrefixSum(j) - PrefixSum(i)`, we calculate range sums in `O(1)` time.

🔹 3. Pattern Recognition

- **Pattern name:** Prefix Sum Hashing (Zero-Delta Tracking).

- **WHEN to use:** Questions asking for constraints on "Subarrays" when the array contains NEGATIVE numbers. (If only positives, use Sliding Window. If negatives, MUST use Prefix Sum Map).
- **HOW to identify:** "Subarray", "Sum equals K", "Sum equals 0".

🔹 4. Solution Evolution

✅ Approach 1: Brute Force

- **Thinking:** Check every single subarray boundary `[i...j]`. If the sum is 0, record length.
- **Complexity:** Time `O(N^2)`, Space `O(1)`.

✅ Approach 2: Optimal Approach (Prefix Sum Map)

- **Best solution idea:** Maintain a running cumulative sum as you iterate left-to-right.
  - Stop and think: Let's say at index `2`, my running sum is `10`. I keep iterating.
  - At index `7`, my running sum is *again* `10`.
  - What does this mathematically prove? It proves that all the numbers I added *between* index `3` and `7` must have completely canceled each other out! The "Delta" between the two states is exactly `0`.
  - **The Algorithm:** As we calculate the running sum, insert `[RunningSum -> CurrentIndex]` into our HashMap.
    1. If `RunningSum == 0`, the subarray from the absolute beginning to right now is a valid 0-sum.
    2. If `RunningSum` already exists in the HashMap, a cycle has occurred. The sequence between the old index and the current index sums exactly to 0. Distance = `CurrentIndex - OldIndex`.

- **Optimal Java Code:**

```java
import java.util.HashMap;

public class LargestSubarrayZeroSum {
    public int maxLenOptimal(int[] nums) {
        // Map stores (Cumulative Sum -> FIRST Index seen)
        HashMap<Integer, Integer> prefixMap = new HashMap<>();

        int maxLength = 0;
        int currentSum = 0;

        for (int i = 0; i < nums.length; i++) {
            currentSum += nums[i];
```

```
            // Case 1: The entire array from index 0 to i sums to 0
            if (currentSum == 0) {
                maxLength = i + 1;
            }

            // Case 2: We have seen this exact cumulative sum before
            if (prefixMap.containsKey(currentSum)) {
                // We do NOT update the map with the new index.
                // We want the LONGEST subarray, so we must keep the oldest,
leftmost index!
                int previousIndex = prefixMap.get(currentSum);
                maxLength = Math.max(maxLength, i - previousIndex);
            } else {
                // Case 3: We have never seen this sum. Record its first
appearance.
                prefixMap.put(currentSum, i);
            }
        }

        return maxLength;
    }
}
```

5. Dry Run

Input: `nums = [15, -2, 2, -8, 1, 7, 10]`

- init: `maxLen = 0`, `currentSum = 0`, map={}

1. `i = 0` (15): `sum = 15`. Map puts `(15 $->$ 0)`.
2. `i = 1` (-2): `sum = 13`. Map puts `(13 $->$ 1)`.
3. `i = 2` (2): `sum = 15`. Map HAS 15!
   - We saw `sum=15` previously at index `0`.
   - Distance: `i(2) - old(0) = 2`. `maxLen = max(0, 2) = 2`.
4. `i = 3` (-8): `sum = 7`. Map puts `(7 $->$ 3)`.
5. `i = 4` (1): `sum = 8`. Map puts `(8 $->$ 4)`.
6. `i = 5` (7): `sum = 15`. Map HAS 15!
   - We saw `sum=15` at index `0` (notice we kept the oldest index!).
   - Distance: `i(5) - old(0) = 5`. `maxLen = max(2, 5) = 5`.
7. Loop ends. Output `5`.

🔹 6. Architectural Thinking

- **WHAT:** Stateful baseline anchoring.
- **WHERE:** Stream aggregation processing.
- **HOW:** Storing the earliest occurrence of a dimensional state allows us to identify cyclical net-zero deviations natively.
- **WHEN:** Hashing the scalar value synchronously immediately blocks redundant index updating.
- **WHO:** This pattern of storing baseline offsets to calculate range differences is identical to Kafka Stream Commits tracking event consumptions!

🔹 7. Edge Cases

- **All zeroes (`[0, 0, 0]`):**
  - `i=0, sum=0` . `max=1` . Map `(0->0)` .
  - `i=1, sum=0` . Map has 0 (at 0). `max=max(1, 1-0) = 1` (Wait... this is wrong!).
  - Ah! Look at the code. `if (currentSum == 0)` runs BEFORE the map checks, and it sets `maxLength = i + 1` . This elegantly catches the 0-sum explicitly from the start, so `maxLength = 2` correctly!

🔹 8. Clean Code

- **Critical Control Flow:** Notice `else { prefixMap.put(...) }` . We explicitly ONLY place the sum into the map if it doesn't already exist. Since we want the *longest* line, overwriting a sum's index with a newer, closer index would physically shorten the resulting sequence.

🔹 9. Interview Training

- **Mistakes to avoid:** A Junior will try to solve this with a Sliding Window (Two Pointers squeezing together). When asked why it fails, immediately explain: "*Sliding Window boundaries operate by growing when the sum is too small, and shrinking when the sum is too big. Negative numbers destroy this logic, because growing the window might actually SHRINK the mathematical sum. Prefix Sum is mathematically mandatory here.*"

🔥 Problem 2: Subarrays with XOR K

🔹 0. Company Tagging

- **Asked in:** Amazon, Google, Meta
- **Frequency:** ⚡ Medium
- **Interview Context:** This is the hardcore bitwise evolution of the Prefix Sum logic. It combines identical Hash Map logic with deeply computer-science specific boolean XOR (`^`) algebra. If

you master this, you prove extreme theoretical competence over raw bytes.

🔹 1. Problem Explanation

Given an array of integers `nums` and an integer `K`, return the total *number* of continuous subarrays whose bitwise XOR evaluates exactly to `K`.

- **Bits Refresher:** XOR (`^`) returns `1` if the bits are different, and `0` if they are the same. A magical property of XOR is that $A ^ A = 0$, and if $A ^ B = C$, then absolutely $A ^ C = B$.
- **Input:** `nums = [4, 2, 2, 6, 4]`, `k = 6`
- **Output:** `4` (The subarrays are `[4, 2]`, `[4, 2, 2, 6, 4]`, `[2, 2, 6]`, and `[6]`).

🔹 2. Data Structure Masterclass

- **DS Used:** `HashMap<Integer, Integer>` (Prefix XOR Frequency Map)
- **Why chosen:** Unlike the "0 Sum" problem that wanted maximum *Length* (storing older Indices), this problem wants total *Count* of valid arrays. The HashMap value must track `Frequency`, not Index!

🔹 3. Pattern Recognition

- **Pattern name:** Algebraic Prefix Frequency Map.
- **WHEN to use:** "Count of Subarrays" combined with "Sum K" or "XOR K".
- **HOW to identify:** Keywords "subarrays evaluate to K", "Total count".

🔹 4. Solution Evolution

✅ Approach 1: Brute Force

- **Thinking:** Double `for` loop, XORing inner ranges continuously.
- **Complexity:** Time `O(N^2)`, Space `O(1)`.

✅ Approach 2: Optimal Approach (Prefix XOR Map)

- **Best solution idea:** The math mirrors the Two-Sum / Prefix-Sum logic completely.
  - Let `currentXor` be the XOR of elements from index 0 to `i`.
  - We want to find some chunk in the history that leaves a remainder of exactly `K`.
  - Equation: `currentXor ^ historyXor = K`.
  - Using XOR rules ($A ^ B = C => A ^ C = B$): `historyXor = currentXor ^ K`.
  - We just need to check: **"How many times have we seen the value `(currentXor ^ K)` in the past?"** Add that frequency to our total answer.

- **Optimal Java Code:**

```java
import java.util.HashMap;

public class SubarraysWithXorK {
    public int countSubarraysOptimal(int[] nums, int k) {
        // Map stores (Cumulative XOR -> Frequency Seen)
        HashMap<Integer, Integer> xorFreqMap = new HashMap<>();

        // Critical base case: The "empty prefix" before the array starts has
an XOR of 0.
        // It has been seen exactly 1 time.
        xorFreqMap.put(0, 1);

        int currentXor = 0;
        int totalCount = 0;

        for (int num : nums) {
            currentXor = currentXor ^ num;

            // Equation: What prefix XOR do we mathematically need to chop off
to leave exactly K?
            int requiredPrefixXor = currentXor ^ k;

            // If we've seen this prefix before, every recorded instance forms
a valid subarray!
            if (xorFreqMap.containsKey(requiredPrefixXor)) {
                totalCount += xorFreqMap.get(requiredPrefixXor);
            }

            // Register this currentXor into history for future iterations to
use
            xorFreqMap.put(currentXor, xorFreqMap.getOrDefault(currentXor, 0) +
1);
        }

        return totalCount;
    }
}
```

5. Dry Run

Input: `nums = [4, 2, 2, 6, 4]`, `K = 6`. Map starts with `{0: 1}`. Total=0.

1. `num=4` : `current = 0^4 = 4` . Req = `4^6 = 2` . Map has 2? No. Add `4` to map $=>$ `{0:1, 4:1}` .
2. `num=2` : `current = 4^2 = 6` . Req = `6^6 = 0` . Map has 0? YES, freq `1` ! Total=1 (subarray `[4, 2]`). Add `6` $=>$ `{0:1, 4:1, 6:1}` .
3. `num=2` : `current = 6^2 = 4` . Req = `4^6 = 2` . Map has 2? No. Add `4` $=>$ `{0:1, 4:2, 6:1}` .
4. `num=6` : `current = 4^6 = 2` . Req = `2^6 = 4` . Map has 4? YES, freq `2` ! Total=1+2=3. (subarray `[2, 2, 6]` and `[4, 2, 2, 6]`). Add `2` $=>$ `{0:1, 4:2, 6:1, 2:1}` .
5. `num=4` : `current = 2^4 = 6` . Req = `6^6 = 0` . Map has 0? YES, freq `1` ! Total=3+1=4. (subarray `[6]`). Add `6` . Result: 4. Correct!

6. Architectural Thinking

- **WHAT:** Look-behind Frequency Accumulation.
- **WHERE:** Stateful iteration relying on Bitwise symmetric properties.
- **HOW:** Storing the occurrences of dimensional states natively acts as a Multiplier—one historical state matching our requirement proves `N` intersecting boundaries.
- **WHEN:** We *must* initialize the baseline `0` state to `1` , otherwise valid subarrays starting precisely at index 0 will mathematically slip through the evaluation logic uncounted.
- **WHO:** XOR logic masking is how Cryptography (specifically stream ciphers and hashing salts) evaluates block permutations rapidly!

7. Edge Cases

- Subarray evaluates straight to `K` spanning from index 0. The initialization `xorFreqMap.put(0, 1)` perfectly isolates this. If `currentXor == K` , then `K ^ K = 0` . Map returns `1` . Perfectly captures it!

8. Clean Code

- `map.put(key, map.getOrDefault(key, 0) + 1)` is the gold standard in Java for updating frequency maps on one pristine line without ugly null-checking blocks.

9. Interview Training

- **Junior Pitfall:** Solving "Largest Subarray 0" by storing the `Index` , and then blindly trying to solve "Count XOR K" by storing the `Index` as well.
- **Senior Check:** "Wait, the interviewer wants TOTAL COUNT, not the physical boundary indices. Storing indices is mathematically useless here. I must flip my HashMap value schema to track raw integer Frequency instead."

🔥 Problem 3: Longest Substring Without Repeating Characters

🔹 0. Company Tagging

- **Asked in:** Amazon, Apple, Meta, ByteDance
- **Frequency:** 🔥 High
- **Interview Context:** This introduces the true **Sliding Window** combined with Hashing. It transitions you from numeric arrays to `String` arrays (characters). This is perhaps the most heavily tested string algorithm on earth.

🔹 1. Problem Explanation

Given a string `s`, find the length of the longest substring without repeating characters.

- **Real-world analogy:** Scanning a streaming log of user IDs. You need the longest continuous block of logs where no user appears twice. Shrinking the block from the left the moment a duplicate enters from the right.
- **Input:** `s = "abcabcbb"`
- **Output:** `3` (The answer is "abc").
- **Constraints:** Solve in strictly `O(N)` time. `O(N^2)` inner-loop character checking fails instantly.

🔹 2. Data Structure Masterclass

- **DS Used:** Sliding Window (`left`, `right` pointers) + `HashMap<Character, Integer>`.
- **Why chosen:** Standard sliding windows expand right and shrink left linearly. By using a HashMap to store the `Last Seen Index` of each character, when a duplicate arrives, we don't need a `while` loop to shrink the window character-by-character... we can lazily jump the `left` pointer instantly past the old duplicate!

🔹 3. Pattern Recognition

- **Pattern name:** Variable Sliding Window (w/ Instant Shrinking).
- **WHEN to use:** Questions asking for constraints on purely *positive/monotonic* data structures without strict fixed lengths (e.g., "Longest continuous without X").
- **HOW to identify:** "Longest substring", "Without repeating".

🔹 4. Solution Evolution

✅ Approach 1: HashSet with Linear Shrink (Standard Sliding Window)

- **Idea:** Two pointers. Right expands. If HashSet contains `char[right]`, start incrementing `left` and removing `char[left]` from the set until `char[right]` is no longer in the set.
- **Complexity:** Time `O(2N) = O(N)`. Space `O(256)`. Completely acceptable logic, but we can do it in exactly 1-pass `O(N)` without the inner `while` loop!

✅ Approach 2: Optimal Approach (Instant Pointer Jumping)

- **Best solution idea:** Use a `HashMap` mapping Character $->$ The Exact Index it was seen at.
  - If we see `'a'` at index `0`, map puts `('a' $->$ 0)`.
  - Say we see `'a'` again at index `3`.
  - Instead of bumping `left` up by 1 slowly... we KNOW the last `'a'` was at index `0`. To fix the duplicate collision, `left` just needs to immediately jump to `LastIndex + 1` (index `1`)!
  - **CRITICAL CHECK:** What if `LastIndex + 1` is *behind* the `left` pointer's current position? We only jump `left` forward. We never let `left` jump backward in time!
- **Optimal Java Code:**

```java
import java.util.HashMap;

public class LongestSubstring {
    public int lengthOfLongestSubstringOptimal(String s) {
        if (s == null || s.length() == 0) return 0;

        // Map stores (Character -> Most recent index it was seen at)
        HashMap<Character, Integer> charIndexMap = new HashMap<>();

        int maxLength = 0;
        int left = 0;

        for (int right = 0; right < s.length(); right++) {
            char currentChar = s.charAt(right);

            // If we have seen this character, and its old index is inside our
current sliding window...
            // ... we must jump the left pointer explicitly past the old index!
            if (charIndexMap.containsKey(currentChar)) {
                // Math.max guarantees 'left' never travels backwards if the
duplicate
                // character was actually found outside (behind) our active
window bounds!
```

```
                left = Math.max(left, charIndexMap.get(currentChar) + 1);
            }

            // Register/overwrite the character's newest index location
            charIndexMap.put(currentChar, right);

            // Calculate the valid window length
            maxLength = Math.max(maxLength, right - left + 1);
        }

        return maxLength;
    }
}
```

5. Dry Run

Input: `"abba"`

- `L=0, R=0` (`'a'`). Map={a:0}. Max=1.
- `L=0, R=1` (`'b'`). Map={a:0, b:1}. Max=2.
- `L=0, R=2` (`'b'`). Collision! Map has 'b' at index 1.
  - `L = max(0, 1 + 1) = 2`.
  - Update map={a:0, b:2}. Length= `2-2+1` = 1. Max remains 2.
- `L=2, R=3` (`'a'`). Collision! Map has 'a' at index 0.
  - `L = max(2, 0 + 1) = max(2, 1) = 2`. (Notice we didn't let `L` jump backwards to 1! The old 'a' is already outside our window safely).
  - Update map={a:3, b:2}. Length= `3-2+1` = 2. Max remains 2.
- Result: 2. Perfectly executed cyclic protection!

6. Architectural Thinking

- **WHAT:** Stateful Window Jumping via Index Caching.
- **WHERE:** Stream aggregation where node constraints break uniquely on individual elements.
- **HOW:** By caching the physical coordinate of the payload, we transform a linear `O(N)` deletion retry-loop into an `O(1)` coordinate transposition.
- **WHEN:** We strictly enforce forward-only temporal traversal (`Math.max(left, ...)`) to prevent resurrecting deleted historical bounds.
- **WHO:** This abstraction mimics TCP Sliding Window frame drops, immediately repositioning the validation pointer past the corrupted bit-frame!

7. Edge Cases

- `" "` (Single space): Handled perfectly as a valid char. Returns 1.
- `"au"` : Fails `contains` instantly. Max evaluates 2. Returns 2.
- **ASCII Optimization:** If the interviewer confirms string is strictly ASCII characters, building a `new int[256]` array and using the character's ASCII integer value as the index is massively faster than the JVM `HashMap` class overhead.

🔹 8. Clean Code

- Instead of maintaining a Set and writing a dirty inner `while(s.charAt(left) != s.charAt(right))` loop, indexing characters natively via Map reduces code branches and mathematically guarantees the minimal number of CPU cycles.

🔹 9. Interview Training

- **Senior Articulation:** When explaining your design, emphasize the `Math.max` mechanism. "A standard Set validation requires `O(2N)` operations because in the worst case, the left pointer has to step individually over every char. By caching the physical Indices in a Map, I mathematically jump the left pointer to safety in strictly 1-pass execution, heavily prioritizing CPU efficiency."

📝 DAILY SUMMARY

🎯 Patterns Learned Today

1. **Mathematical Prefix Maps:** Leveraging standard variables as commutative history accumulators `(PrefixSum(j) - PrefixSum(i)) = Delta` to calculate continuous range matrices in `O(1)` logic loops.
2. **Value Schema Evolution:** Learning that HashMap *Values* must change based on Output Requirements. If asking for physical `Length`, store `(Key $->$ Index)`. If asking for mathematical `Total Count`, store `(Key $->$ Frequency)`.
3. **Sliding Window Jumping:** Combining monotonically expanding window boundaries with Dictionary memory mappings to teleport the trailing pointer instantaneously past collision events, erasing nested loop bounds.

⚖️ Decision Rules

- If dealing with **Subarray Constraints** WITH NEGATIVE NUMBERS $=>$ Discard Sliding Windows. Deploy Prefix Sum Maps natively.
- If finding **Maximum Lengths** inside Hash Maps $=>$ DO NOT overwrite older map keys! You want the earliest index possible.

- If finding **String Subsequence Uniqueness** $=> `Map Character` ->$ Index and deploy `Math.max(L, oldIndex + 1)` teleporting.

⚠ Key Mistakes to Avoid

- **Longest 0-Sum Subarray:** Forgetting that if `currentSum` evaluates exactly to the required target natively, the length spans from absolute 0 to the current `i`. Explicitly validating this before the Map lookup is critical.
- **Longest Substring without Repeat:** Creating a Map collision and carelessly executing `left = map.get(char) + 1` without validation. If the old character sits physically *behind* the current sliding window, `left` will travel backwards in time, destroying the local sequence!

See you tomorrow for Day 6: Array Math & Majority Algorithms!

Day 6: Arrays - Math & Majority Elements

Welcome to Day 6! Today we delve into **Algorithmic Annihilation** and **Combinatorics**. You will learn Boyer-Moore's Voting Algorithm—one of the most elegant `O(1)` space algorithms ever invented for data stream processing. We will also revisit spatial mathematics to solve 2D grid pathing without using recursion or Dynamic Programming matrices.

🔥 Problem 1: Majority Element (`> N/2`)

🔹 0. Company Tagging

- **Asked in:** Amazon, Google, Microsoft, Apple
- **Frequency:** 🔥 High
- **Interview Context:** This is the ultimate "Do you know the trick?" question. A Junior uses a HashMap to count frequencies (`O(N)` space). A Senior uses Moore's Voting Algorithm to identify the absolute majority in strictly `O(1)` space using a single pass.

🔹 1. Problem Explanation

Given an array `nums` of size `n`, return the majority element. The majority element is the element that appears **strictly more than** `n / 2` times. You may assume that the majority element always exists in the array.

- **Real-world analogy:** A political election. If one candidate has *more* than 50% of the total votes, even if every other candidate teams up against them, the majority candidate will still have votes left over.
- **Input:** `nums = [2, 2, 1, 1, 1, 2, 2]`
- **Output:** `2`
- **Constraints:** Must execute in `O(N)` time and strictly `O(1)` space.

🔹 2. Data Structure Masterclass

- **DS Used:** Primitive track variables (`majorityElement`, `count`).
- **Why chosen:** Required by `O(1)` space limits.
- **Time complexity benefits:** We don't allocate Memory Heaps for HashMaps, meaning zero JVM Garbage Collection overhead during iteration.

🔹 3. Pattern Recognition

- **Pattern name:** Boyer-Moore Majority Voting Algorithm.

- **WHEN to use:** When asked to find an element appearing more than `N/2` times.
- **HOW to identify:** "Majority Element", "more than n/2".

🔹 4. Solution Evolution

✅ Approach 1: Brute Force / HashMap

- **Thinking:** Create a `HashMap<Integer, Integer>`. Count the frequency of every element. Iterate through the map and return the element with `freq > n/2`.
- **Complexity:** Time `O(N)`, Space `O(N)`. Highly inefficient for memory-constrained distributed systems analyzing millions of IP traffic logs.

✅ Approach 2: Sorting

- **Thinking:** If an element appears more than `N/2` times, and the array is SORTED, it will unquestionably occupy the exact middle index `nums[n/2]`, regardless of whether it's the smallest, largest, or middle physical value.
- **Code:** `Arrays.sort(nums); return nums[nums.length / 2];`
- **Complexity:** Time `O(N log N)`, Space `O(1)`.

✅ Approach 3: Optimal Approach (Boyer-Moore Voting)

- **Best solution idea:** The "Annihilation" Principle.
  - Assume the first element is the majority candidate. Give it a `count` of 1.
  - If we see the SAME element again, `count++` (our candidate is getting stronger).
  - If we see a DIFFERENT element, `count--` (a rival voter cancels out one of our candidate's votes!).
  - If `count` hits `0`, our candidate was entirely annihilated. The very next element we see becomes the NEW candidate, starting at `count = 1`.
  - **Mathematical Guarantee:** Because the true majority element exists *more than* `N/2` times, it physically cannot be annihilated by the minority elements. It will always survive at the end!
- **Optimal Java Code:**

```java
public class MajorityElement {
    public int majorityElementOptimal(int[] nums) {
        int candidate = 0;
        int count = 0;

        // Phase 1: Annihilation / Voting
        for (int num : nums) {
            if (count == 0) {
```

```
                // Previous candidate was annihilated. Crown a new one!
                candidate = num;
                count = 1;
            } else if (num == candidate) {
                // Same candidate, bolster defensive count
                count++;
            } else {
                // Rival candidate, mutual annihilation
                count--;
            }
        }

        // NOTE: The problem guarantees a majority element exists.
        // If it didn't, we would need a Phase 2 here to loop the array
        // one more time and explicitly verify that 'candidate' freq > N/2.
        return candidate;
    }
}
```

5. Dry Run

Input: `nums = [7, 7, 5, 7, 5, 1, 5, 7, 5, 5, 7, 7, 7, 7, 7]`

- To save space, let's dry run `[2, 2, 1, 1, 1, 2, 2]`

1. `num = 2` : `count` is 0 $=>$ `candidate=2` , `count=1` .
2. `num = 2` : Matches! `count=2` .
3. `num = 1` : Rival! `count=1` .
4. `num = 1` : Rival! `count=0` . (Candidate `2` has been temporarily overthrown).
5. `num = 1` : `count` is 0 $=>$ `candidate=1` , `count=1` . (New king).
6. `num = 2` : Rival! `count=0` (Candidate `1` overthrown).
7. `num = 2` : `count` is 0 $=>$ `candidate=2` , `count=1` . (King returns). Result: `2` survives!

6. Architectural Thinking

- **WHAT:** Destructive interference state-tracking.
- **WHERE:** Stream processing where memory bounds must remain stringently `O(1)` .
- **HOW:** Allowing data packets to mutually destroy each other's weight metrics unearths the dominant recurring signal.
- **WHEN:** Checked sequentially, keeping only the surviving scalar value payload.
- **WHO:** This logic is deployed natively in Distributed Sensor Networks to determine the consensus reading from noisy hardware monitors!

🔹 7. Edge Cases

- **Array of length 1 (`[5]`):** `count=0` , `candidate=5` . Loops ends. Correct.
- **Problem variation (No Guarantee):** If the interviewer removes the "guaranteed to exist" clause, returning `candidate` blindly is a fatal bug. Example: `[1, 2, 3]` . `candidate` finishes as `3` . You MUST add a second `for()` loop to count occurrences of `candidate` and assert `freq > N/2` .

🔹 8. Clean Code

- Avoid convoluted ternary operators for the 3 distinct states (Empty, Match, Mismatch). Explicit `if / else if / else` proves your logical branching to the interviewer.

🔹 9. Interview Training

- **Senior Articulation:** "The algorithm operates on the principle of mutual destruction. Since a majority element occupies more than half the array, even in the absolute worst-case dispersion where every non-majority element strictly deducts from the majority's count, the majority's count will mathematically evaluate to at least `+1` at termination."

🔥 Problem 2: Majority Element II (`> N/3`)

🔹 0. Company Tagging

- **Asked in:** Google, Amazon, Meta
- **Frequency:** ⚡ Medium
- **Interview Context:** If you pass Moore's Voting for `N/2` , they will hit you immediately with `N/3` . Can you dynamically scale an algorithm? How many elements can physically appear `> N/3` times? Exactly **TWO**. You must scale Moore's algorithm to track two simultaneous candidates.

🔹 1. Problem Explanation

Given an integer array of size `n` , find all elements that appear **strictly more than** `n/3` times.

- **Mathematical constraint:** In an array of 10 elements, `N/3 = 3` . You need elements appearing $>= 4$ times. How many elements can fit? `4+4=8` . A third element would require `12` spots! So, there is a maximum of **2** valid majority candidates.
- **Input:** `nums = [1, 1, 1, 3, 3, 2, 2, 2]` (`N=8` , target `>2`)
- **Output:** `[1, 2]`

- **Constraints:** O(N) time and O(1) space.

🔹 2. Data Structure Masterclass

- **DS Used:** Two Primitive track variables (`cand1` , `cand2` , `count1` , `count2`).
- **Why chosen:** Again, strictly complying with the `O(1)` spacial requirement by explicitly declaring exactly 4 variables.

🔹 3. Pattern Recognition

- **Pattern name:** Extended Boyer-Moore Voting Algorithm.
- **WHEN to use:** "> N/3 Majority Element" searches.

🔹 4. Solution Evolution

✅ Approach 1: HashMap

- **Thinking:** Same as before. Count everything. Iterate keys, if `freq > n/3` , add to result list.
- **Complexity:** Time `O(N)` , Space `O(N)` . Fails the spatial constraint.

✅ Approach 2: Optimal Approach (Double Moore Voting)

- **Best solution idea:** Track two candidates.
  - If `num == cand1` , `count1++` .
  - If `num == cand2` , `count2++` .
  - If `count1 == 0` , assign `cand1 = num` .
  - If `count2 == 0` , assign `cand2 = num` .
  - **The Annihilation Rule:** If `num` matches NEITHER candidate, it is a rebel third party! It attacks BOTH candidates. `count1--` AND `count2--` .
  - Because the problem does NOT guarantee existence, a Phase 2 manual frequency check is strictly required.
- **Optimal Java Code:**

```java
import java.util.ArrayList;
import java.util.List;

public class MajorityElementTwo {
    public List<Integer> majorityElementOptimal(int[] nums) {
        int cand1 = Integer.MIN_VALUE, cand2 = Integer.MIN_VALUE;
        int count1 = 0, count2 = 0;

        // Phase 1: Dual Annihilation
        for (int num : nums) {
            if (cand1 == num) {
```

```java
                count1++;
            } else if (cand2 == num) {
                count2++;
            } else if (count1 == 0) {
                cand1 = num;
                count1 = 1;
            } else if (count2 == 0) {
                cand2 = num;
                count2 = 1;
            } else {
                // A third distinct element appears! Annihilate both leaders!
                count1--;
                count2--;
            }
        }

        // Phase 2: Verification (Strictly mandatory for N/3)
        List<Integer> result = new ArrayList<>();
        int freq1 = 0, freq2 = 0;

        for (int num : nums) {
            if (num == cand1) freq1++;
            else if (num == cand2) freq2++;
        }

        int n = nums.length;
        if (freq1 > n / 3) result.add(cand1);
        if (freq2 > n / 3) result.add(cand2);

        return result;
    }
}
```

5. Dry Run

Input: `nums = [1, 2, 2, 3, 2, 1, 1, 3]`

1. (1): `cand1 = 1`, `c1 = 1`.
2. (2): `cand2 = 2`, `c2 = 1`.
3. (2): `cand2` matches! `c2++` (2).
4. (3): Matches neither! `c1=0, c2=1`. (Wait! `cand1` was zeroed!).
5. (2): `cand2` matches! `c2++` (2).
6. (1): Matches neither? No, `cand1` is still `1`, but its count is `0`. The `else if (count1 == 0)` fires! `cand1=1, c1=1`.

7. (1): `cand1` matches! `c1++` (2).
8. (3): Matches neither! `c1--` (1), `c2--` (1).

- **Candidates:** `cand1 = 1`, `cand2 = 2`.
- **Phase 2 (Counting):** `1` appears 3 times. `2` appears 3 times. `N=8`. Limit = `8/3 = 2`.
- `3 > 2`. Both are valid! Output `[1, 2]`.

🔹 6. Architectural Thinking

- **WHAT:** Multi-node consensus protocols.
- **WHERE:** Stream processing for fractional dominance thresholds.
- **HOW:** Extending the mathematical proof that `K` candidates require `K` trackers and simultaneously degrade under `K`-way node collisions.
- **WHO:** Used in Byzanatine Fault Tolerance protocols in block-chains, verifying if $>33%$ of validator nodes agree on a corrupted hash without buffering the whole chain!

🔹 7. Edge Cases

- **Initialization Danger:** If the array contains `Integer.MIN_VALUE`, initializing candidates to `MIN_VALUE` without counting logic protection will trigger false positives! Fortunately, the `count` conditionals execute before arbitrary candidate value overwrites, buffering us from this hazard.

🔹 8. Clean Code

- **Structure:** In Phase 2, `if (num == cand1)` followed by `**else** if (num == cand2)` is critical. If `cand1 == cand2` (due to some edge-case initialization flaw), avoiding double-counting the frequency ensures mathematically pure results.

🔹 9. Interview Training

- **Junior Pitfall:** Trying to decrement only ONE counter instead of BOTH counters during a Third-Party collision. If `A`, `B`, and `C` are competing, a vote for `C` mathematically hurts **both** `A`'s and `B`'s percentage threshold simultaneously!

🔥 Problem 3: Grid Unique Paths

🔹 0. Company Tagging

- **Asked in:** Google, Amazon, Microsoft
- **Frequency:** 🔥 High

- **Interview Context:** This is typically asked as a Dynamic Programming question. The interviewer wants to see you write the 2D DP matrix. However, an elite Senior engineer recognizes that this is purely **Combinatorics Math** (Permutations) and solves it in `O(N)` time with `O(1)` space without creating a DP grid!

🔹 1. Problem Explanation

There is a robot on an $m x n$ grid. The robot is initially located at the **top-left corner** (i.e., `grid[0][0]`). The robot tries to move to the **bottom-right corner** (`grid[m - 1][n - 1]`). The robot can only move **either down or right** at any point in time. Return the number of possible unique paths.

- **Input:** `m = 3` , `n = 7`
- **Output:** `28`
- **Constraints:** Solve mathematically for Senior rating.

🔹 2. Data Structure Masterclass

- **DS Used:** None (Pure Mathematics).
- **Why chosen:** Standard `O(M x N)` recursive DP requires caching matrices. Math requires purely `O(1)` primitives.

🔹 3. Pattern Recognition

- **Pattern name:** Pascal's Combinatorics (`nCr`).
- **WHEN to use:** "Count total paths" moving strictly Right/Down in an unblocked grid.
- **HOW to identify:** "Unique paths", "m x n grid".

🔹 4. Solution Evolution

✅ Approach 1: Recursion (Disaster)

- **Thinking:** From `(0, 0)` , explore `(1, 0)` and `(0, 1)` . Recursively sum paths.
- **Complexity:** Time `O(2^{N x M})` Exponential explosion. Space `O(\text{Call Stack})` . Fails instantly on grids > 15x15.

✅ Approach 2: Dynamic Programming (Standard Better)

- **Thinking:** Create a 2D matrix `dp[m][n]` . Every cell's path count is `dp[up] + dp[left]` .
  - Base case: Top row and Leftmost column are all `1` s.
  - Iterate: `dp[i][j] = dp[i-1][j] + dp[i][j-1]`
- **Complexity:** Time `O(M x N)` , Space `O(M x N)` . (This is generally acceptable, but we can do far better).

✅ Approach 3: Optimal Approach (Mathematical `nCr`)

- **Best solution idea:** Look at a $3 x 7$ grid.
  - To get from Start to End, how many times MUST we move physically `Down` (D)? To get to row 3, we MUST move down exactly `m-1 = 2` times.
  - How many times MUST we move `Right` (R)? `n-1 = 6` times.
  - Every valid path is just a scrambled string of exactly `2 D's` and `6 R's`. (e.g., `D-R-R-D-R-R-R-R`).
  - Total moves = `2 + 6 = 8`.
  - The mathematical question becomes: **"Out of 8 available step slots, how many unique ways can I choose exactly 2 slots to be my 'Down' steps?"**
  - That is `8C2` combinations!
  - Formula: $\text{Total Steps} = M - 1 + N - 1 = M + N - 2$.
  - $\text{Choose} = \text{smaller of} (M-1, N-1)$.
  - $^nC_r = \text{ans} x (n - \text{col}) / \text{col}$.
- **Optimal Java Code:**

```java
public class UniquePaths {
    public int uniquePathsOptimal(int m, int n) {
        // Total physical steps required to reach the destination
        int N = m + n - 2;

        // We can choose EITHER the 'Down' steps or 'Right' steps.
        // Picking the smaller one shrinks our iteration timeframe!
        int R = Math.min(m - 1, n - 1);

        long ans = 1; // Use long to prevent multiplication overflow integer
breaking

        // Calculate nCr math
        // Example: 8C2 = (8 * 7) / (1 * 2) = 28
        for (int i = 1; i <= R; i++) {
            ans = ans * (N - R + i);
            ans = ans / i;
        }

        return (int)ans;
    }
}
```

5. Dry Run

Input: `m=3`, `n=7`.

- `N = 3 + 7 - 2 = 8`.
- `R = min(2, 6) = 2`.
- Loop 1 (`i=1`): `ans = 1 * (8 - 2 + 1) = 7`. Wait, no.
  - Let's do the standard `N` descending form instead to keep it simple natively.
  - Code: `ans * (N - R + i)` is mathematically valid, but easier notation:
  - `ans = ans * (N - i + 1) / i;`
  - Loop 1 (`i=1`): `ans = 1 * (8 - 1 + 1) / 1 = 8`.
  - Loop 2 (`i=2`): `ans = 8 * (8 - 2 + 1) / 2 = 8 * 7 / 2 = 28`.
- Result is 28. Perfect!

(Note: In the code provided above, the loop iterates `ans * (N - R + i)`, which effectively computes the numerator backwards `(8-2+1 = 7, 8-2+2 = 8)`. Both versions calculate `nCr` accurately, but the descending `N` is slightly more intuitive).

6. Architectural Thinking

- **WHAT:** Compressing 2D state matrices entirely into a declarative combinatorics formula.
- **WHERE:** O(1) mathematical execution.
- **HOW:** Relying on physical dimensional constraints (you *must* move right `X` times) to prove that temporal pathing variations are just permutation anagrams!
- **WHEN:** We execute division natively *inside* the loop sequentially. `(8*7)/(1*2)`. This guarantees no massive intermediate floating-point overflow!
- **WHO:** Converting iterative state structures to combinatorial equations is the foundation of high-performance physics rendering engines minimizing compute loops!

7. Edge Cases

- **1x1 Grid:** `N = 0`, `R = 0`. Loop skips. Returns `1`. Correct (1 path, standing still).
- **1xN Grid:** `R = min(0, n) = 0`. Loop skips. Returns `1`. Correct (just a straight line).

8. Clean Code

- `long ans = 1`. If `N` is large, multiplying `ans * N` will shatter the 32-bit Java `int` limit before the division slash can shrink it back down! You must utilize 64-bit `long` internally to safely calculate the numerator.

9. Interview Training

- **The Optimal Stratagem:** When asked Grid Unique Paths, ALWAYS start by writing the `O(M x N)` DP solution first. This proves you understand Matrix mapping and base cases.
- *After* writing it, stop and say: "*I notice a pattern. Every cell is the exact sum of the cell above and left of it. This is literally generating Pascal's Triangle diagonally. Because of that, I can discard this entire DP matrix and calculate the final destination cell using* `nCr` *Combinatorics in* `O(N)` *time.*" The interviewer will be blown away.

📝 DAILY SUMMARY

🎯 Patterns Learned Today

1. **Moore's Voting Algorithm:** Solving fractional subset queries in `O(1)` spacial complexity via localized aggressive annihilation mechanics.
2. **Double Moore Validation:** Expanding the framework to handle `>N/3` counts using isolated multi-trackers, highlighting the strict requirement for Phase-2 secondary assertion checks.
3. **Combinatorial DP Eradication:** Identifying when recursive state maps (like Grid Paths) natively produce Pascal's Triangle structures, allowing complete replacement with singular `nCr` algebra loops.

⚖️ Decision Rules

- If dealing with **Majority Elements** representing distinct physical entities $=>$ Utilize **Annihilation Voting**. Reject HashMaps.
- If evaluating **Path Options across an empty matrix** $=>$ Do not execute pathfinding algorithms (DFS/DP). Compute dynamically using `nCr` factorizations.

⚠ Key Mistakes to Avoid

- `N/3` **Majority:** Failing to reset candidates correctly. The `else if` chaining must sequentially test if `count1` is 0 BEFORE testing if `count2` is 0, to prevent a single new element from simultaneously overwriting both tracking slots!
- **Grid Paths** `nCr` **:** Declaring `ans` as a standard `int`. Calculating permutations creates monstrous intermediate integers. Without `long`, your logic is flawlessly correct but will generate negative overflow garbage output.

See you tomorrow for Day 7: Two Pointer Advanced, and our Weekly Revision Guide!

Day 7: Two Pointer Advanced & Architectural Arrays

Welcome to Day 7, the final day of Week 1! Today we conquer the pinnacle of Array manipulation: Multi-layered Pointer architectures and Divide & Conquer math. *Trapping Rain Water* is arguably the hardest "pure array" problem deployed by FAANG. *Reverse Pairs* introduces you to the concept of piggy-backing algorithmic logic onto pre-existing Sorting frameworks (Merge Sort) to achieve mathematical impossibilities.

🔥 Problem 1: Remove Duplicates from Sorted Array

🔹 0. Company Tagging

- **Asked in:** Microsoft, Apple, Meta
- **Frequency:** ⚡ Medium
- **Interview Context:** This problem is a fundamental warmup. It explicitly tests if you understand how to mutate an array in-place without allocating a secondary buffer list.

🔹 1. Problem Explanation

Given an integer array `nums` sorted in non-decreasing order, remove the duplicates **in-place** such that each unique element appears only once. The relative order of the elements should be kept the same. Return the number of unique elements `k`.

- **Real-world analogy:** Compacting a fragmented hard-drive partition. You move the valid data sequentially to the front, completely ignoring the garbage data left behind at the end of the partition.
- **Input:** `nums = [0, 0, 1, 1, 1, 2, 2, 3, 3, 4]`
- **Output:** `5` (The array is modified to `[0, 1, 2, 3, 4, _, _, _, _, _]`).
- **Constraints:** `O(1)` extra space.

🔹 2. Data Structure Masterclass

- **DS Used:** Raw Array + 2 Pointers (`Slow` and `Fast`).
- **Why chosen:** Required for `O(1)` spacial limits.
- **Internal working:** The `Slow` pointer tracks the boundary of the "Unique, Valid" array. The `Fast` pointer acts as a scout, continuously scanning into the unknown right-hand side.

🔹 3. Pattern Recognition

- **Pattern name:** Fast & Slow Pointers (In-place Compaction).
- **WHEN to use:** "Remove in-place", "Compact array", "Sorted array duplicates".

🔹 4. Solution Evolution

✅ Approach 1: HashSet (Brute Force)

- **Thinking:** Dump the array into a `LinkedHashSet` (to preserve order while killing duplicates). Unload the Set back into the array from index 0.
- **Complexity:** Time `O(N)`, Space `O(N)`. Fails the `O(1)` space constraint instantly.

✅ Approach 2: Optimal Approach (Two Pointers)

- **Best solution idea:** Both pointers start at the beginning.
  - The first element `nums[0]` is naturally unique. So our `Slow` pointer (the writer) stays at index `0`.
  - Let our `Fast` pointer (the scanner) go out and look for the next distinct element.
  - While `nums[Fast] == nums[Slow]`, it's a duplicate. Ignore it. `Fast++`.
  - The moment `nums[Fast] != nums[Slow]`, we found a new unique number!
  - We must save it! We increment `Slow` (to expand our unique territory) and then WRITE the new number over the old garbage: `nums[Slow] = nums[Fast]`.
- **Optimal Java Code:**

```java
public class RemoveDuplicates {
    public int removeDuplicatesOptimal(int[] nums) {
        if (nums.length == 0) return 0;

        // 'insertIndex' acts as the Slow Pointer. It demarcates the boundary
of the unique array.
        // It starts at 0 because nums[0] is inherently the first unique
element.
        int insertIndex = 0;

        // 'i' acts as the Fast Pointer traversing the entire array.
        for (int i = 1; i < nums.length; i++) {
            // If the scout finds a number different from the LAST unique
number we locked in...
            if (nums[i] != nums[insertIndex]) {
                // Expand our valid territory boundary
                insertIndex++;
                // Overwrite the garbage data at the new boundary with the
found unique number
                nums[insertIndex] = nums[i];
            }
```

```
        }

        // The array is 0-indexed. If insertIndex is at index 4, there are 5
unique elements.
        return insertIndex + 1;
    }
}
```

5. Dry Run

Input: `nums = [1, 1, 2]`

- Init: `insertIndex = 0`.
- Loop `i = 1`: `nums[1] (1) == nums[0] (1)`. Match! Duplicate. Loop skips, `i` becomes `2`.
- Loop `i = 2`: `nums[2] (2) != nums[0] (1)`. New element!
  - `insertIndex++` becomes `1`.
  - `nums[1] = nums[2]` $=>$ `nums[1] = 2`. array is `[1, 2, 2]`.
- Loop ends. Returns `insertIndex + 1` = `2`. (The first 2 elements `[1, 2]` are perfectly unique).

6. Architectural Thinking

- **WHAT:** Destructive contiguous compaction.
- **WHERE:** Linear read-write operations inside a singular memory block.
- **HOW:** Because the `Fast` read-head strictly moves equal-to or faster than the `Slow` write-head, it mathematically guarantees we will *never* overwrite data we haven't read yet!
- **WHO:** This precise loop is exactly how the OS Defragmentation routines compact logical disk blocks!

7. Edge Cases

- **Empty Array (`[]`):** Guard clause `if length == 0 return 0` handles the `NullPointer` / `IndexOutOfBounds` risk perfectly.
- **Fully Unique Array (`[1, 2, 3]`):** `insertIndex` increments synchronously with `i`. The code executes `nums[1] = nums[1]`, which is slightly redundant computation, but writing an `if (i != insertIndex)` check is actually slower due to CPU branch prediction failures. The redundant write is optimal!

8. Clean Code

- Notice we don't return the array itself; we return the `Count`. In C/Java, you cannot physically resize a native integer array block. Returning the valid bounding length allows the caller function to know exactly which subset of the array to read.

🔹 9. Interview Training

- **Mistakes to avoid:** A Junior will desperately try to pull the right-side elements to the left by writing nested `for` loops shifting `N` elements every time a duplicate is found resulting in catastrophic `O(N^2)` time. Remember: you don't need to "delete" the duplicates, you just "overwrite" them and update your boundary pointer!

🔥 Problem 2: Trapping Rain Water

🔹 0. Company Tagging

- **Asked in:** Amazon, Google, Meta, Bloomberg
- **Frequency:** 🔥 High
- **Interview Context:** This is the ultimate boss of the Two Pointer paradigm. It tests spatial reasoning. Can you identify the strict bounding constraints (Left Wall vs Right Wall) that physically allow a 2D volumetric state (Water) to exist?

🔹 1. Problem Explanation

Given `n` non-negative integers representing an elevation map where the width of each bar is 1, compute how much water it can trap after raining.

- **Real-world analogy:** Pouring water into an uneven bowl. For water to physically pool at any specific column, there MUST be a taller pillar somewhere to its left, AND a taller pillar somewhere to its right. The water level is bottlenecked entirely by whichever of those two enclosing walls is *shorter*.
- **Input:** `height = [0,1,0,2,1,0,1,3,2,1,2,1]`
- **Output:** `6`

🔹 2. Data Structure Masterclass

- **DS Used:** Two Pointers (`left` and `right`) + Bounding trackers (`leftMax`, `rightMax`).
- **Why chosen:** Required for `O(1)` space.

🔹 3. Pattern Recognition

- **Pattern name:** Bottleneck Convergence (Dual Peak Pointers).

- **WHEN to use:** Questions asking for Area, Volume, "Trapping", or "Largest Rectangle" utilizing variable heights.

🔹 4. Solution Evolution

✅ Approach 1: Brute Force

- **Thinking:** For every single index `i`, run a loop to the left to find its absolute highest peak. Run a loop to the right to find its absolute highest peak. Water at exactly `i` = `Math.min(maxLeft, maxRight) - height[i]`.
- **Complexity:** Time `O(N^2)`, Space `O(1)`.

✅ Approach 2: Better Approach (Prefix/Suffix Arrays)

- **Idea:** Pre-compute the highest left and highest right walls!
  - `leftMaxArray[]` tracks the max height seen from $0 -> i$.
  - `rightMaxArray[]` tracks the max height seen from $N-1 -> i$.
  - Loop one final time: `water += Math.min(leftMax[i], rightMax[i]) - height[i]`.
- **Complexity:** Time `O(3N) = O(N)`. Space `O(2N) = O(N)`. Fantastic, but the interviewer will demand `O(1)` space.

✅ Approach 3: Optimal Approach (Two Pointers)

- **Best solution idea:** The formula is strictly `min(leftMax, rightMax)`.
  - Put a pointer at the absolute `left` and `right` sides of the array. Track `leftMax` and `rightMax`.
  - If `leftMax < rightMax`, we know with 100% mathematical certainty that the bottleneck for the `left` pointer relies purely on `leftMax`. Even if there is a 500ft wall hiding somewhere in the chaotic middle of the array, `leftMax` enforces a strict ceiling. Therefore, we can confidently calculate the water trapped exactly at `left`, and move `left++`!
  - If `rightMax < leftMax`, the right side is the bottleneck. Calculate water at `right`, and move `right--`!
- **Optimal Java Code:**

```java
public class TrappingRainWater {
    public int trapOptimal(int[] height) {
        if (height == null || height.length <= 2) return 0;
```

```
        int left = 0;
        int right = height.length - 1;

        int leftMax = 0;
        int rightMax = 0;

        int totalWater = 0;

        while (left < right) {
            // The LEFT side is the structural bottleneck
            if (height[left] <= height[right]) {
                // Have we found a new structural wall?
                if (height[left] >= leftMax) {
                    leftMax = height[left];
                } else {
                    // Current block is smaller than the wall. It traps water
to the top of the wall!
                    totalWater += leftMax - height[left];
                }
                left++;
            }
            // The RIGHT side is the structural bottleneck
            else {
                // Have we found a new structural wall?
                if (height[right] >= rightMax) {
                    rightMax = height[right];
                } else {
                    totalWater += rightMax - height[right];
                }
                right--;
            }
        }

        return totalWater;
    }
}
```

5. Dry Run

Input: `[0, 1, 0, 2, 1, 0, 1, 3]`

- `L=0` (0), `R=7` (3). `lMax=0`, `rMax=0`. `total=0`.
- `L(0) <= R(3)`: `botL`. `h[L] >= lMax` ($0 >= 0$). `lMax=0`. `L++` (1).
- `L(1) <= R(3)`: `botL`. `h[L] >= lMax` ($1 >= 0$). `lMax=1`. `L++` (2).
- `L(2) <= R(3)`: `botL`. `h[L] < lMax` ($0 < 1$). Water += `1 - 0` = 1. `L++` (3).

- `L(3) <= R(3)` : `botL` . `h[L] >= lMax` ($2 >= 1$). `lMax=2` . `L++` (4).
- `L(4) <= R(3)` : `botL` . `h[L] < lMax` ($1 < 2$). Water += `2 - 1` = 1. (Total=2). `L++` (5).
- `L(5) <= R(3)` : `botL` . `h[L] < lMax` ($0 < 2$). Water += `2 - 0` = 2. (Total=4). `L++` (6).
- `L(6) <= R(3)` : `botL` . `h[L] < lMax` ($1 < 2$). Water += `2 - 1` = 1. (Total=5). `L++` (7).
- `L` and `R` collide at index 7. Loop breaks. Output = 5. (Wait, is the result exactly 5 for this chunk? Yes, my manual dry run logic executed flawlessly on the sub-section provided!)

6. Architectural Thinking

- **WHAT:** Simultaneous bidirectional convergence exploiting variable ceilings.
- **WHERE:** Executed entirely iteratively with `O(1)` memory.
- **HOW:** By definitively establishing whichever side is structurally weaker (`height[left] <= height[right]`), we deduce that any internal chaotic spikes are mathematically irrelevant to the capacity of the weaker node, allowing `O(1)` calculations.
- **WHEN:** We calculate incrementally on every inward movement step.
- **WHO:** This localized bottleneck calculation translates cleanly into Supply Chain route optimizations, figuring out the maximum throughput constraints across divergent graph node pipelines.

7. Edge Cases

- **Flat terrain / Decreasing terrain (`[5, 4, 3]` , `[1, 1, 1]`):** The `leftMax` and `rightMax` continuously update and overwrite without the `else` block ever firing. Output is cleanly `0` trapped water.

8. Clean Code

- **Defensive Check:** `if (height.length <= 2) return 0;` . You physically cannot trap water with only 2 pillars. This simple line saves arbitrary null pointer faults effortlessly.

9. Interview Training

- **Senior Check:** To prove your mastery, explain exactly *why* this works to the interviewer. "*Because `MIN(leftMax, rightMax)` governs the water level, checking `height[left] <= height[right]` ensures that `leftMax` acts as the strict bottleneck. Even if there are infinitely tall walls further into the center, they cannot raise the water level at my current `left` position beyond `leftMax` .*"

🔥 Problem 3: Reverse Pairs (Divide & Conquer)

🔹 0. Company Tagging

- **Asked in:** Google, Amazon, Meta
- **Frequency:** ❄️ Hard/Rare (Usually for L5+ Roles)
- **Interview Context:** This is fundamentally a Merge Sort algorithms test. A Junior reads "Pairs" and assumes Two Pointers. A Senior reads "Pairs where `i < j` and `nums[i] > 2 * nums[j]` " and immediately recognizes the properties of Inversion Counting via Divide & Conquer sorting logic.

🔹 1. Problem Explanation

Given an integer array `nums` , return the number of reverse pairs. A reverse pair is a pair `(i, j)` where `0 <= i < j < nums.length` and `nums[i] > 2 * nums[j]` .

- **Analogy:** Ranking algorithmic speeds. We are trying to find instances where the trailing array element is less than *half* the size of the preceding element.
- **Input:** `nums = [1,3,2,3,1]`
- **Output:** `2` (Pairs are `(1) 3 > 2*1` and `(3) 3 > 2*1`).

🔹 2. Data Structure Masterclass

- **DS Used:** Merge Sort Recursion Tree + Temporary array allocation.
- **Why chosen:** Traversing all pairs is `O(N^2)` . But if we divide the array in half, and sort the halves... we can linearly `O(N)` count the reverse pairs between the Left-Half and Right-Half identically during the Merge Step, resulting in an `O(N log N)` execution!

🔹 3. Pattern Recognition

- **Pattern name:** Modified Merge Sort (Inversion Counting).
- **WHEN to use:** "Count pairs where $A[i] > K * A[j] `with` i < j$".

🔹 4. Solution Evolution

✅ Approach 1: Brute Force

- **Thinking:** Double loops! Note: $!nums[i] > 2 * nums[j]$ requires casting to `(long)` to prevent $2 x MAX_VALUE$ integer overflow logic bombs!
- **Complexity:** Time `O(N^2)` , Space `O(1)` . TLE guarantee.

✅ Approach 2: Optimal Approach (Merge Sort Inversion Checking)

- **Best solution idea:** Piggyback on `MergeSort()` .

- Standard MergeSort recursively divides the array into Left and Right halves, then merges them in sorted order.
- Assume the Left Half `[4, 6, 8]` is sorted. Assume Right Half `[1, 2, 3]` is sorted.
- Before we merge them, perform the Reverse Pair check!
- Iterate `i` over the Left Half. Iterate `j` over the Right Half.
- If `Left[i] > 2 * Right[j]`, then *every element to the right of* `i` *in the Left Half will ALSO be greater than* `2 * Right[j]` because the Left Half is already sorted! No need to check them individually! Just add `(mid - i + 1)` to the total count and move `j` forward!

- **Optimal Java Code:**

```java
public class ReversePairs {
    public int reversePairs(int[] nums) {
        if (nums == null || nums.length < 2) return 0;
        return mergeSort(nums, 0, nums.length - 1);
    }

    // Core Divide & Conquer logic
    private int mergeSort(int[] nums, int left, int right) {
        if (left >= right) return 0;

        int mid = left + (right - left) / 2;
        int count = 0;

        // Sum inversions from left half + right half + cross inversions
        count += mergeSort(nums, left, mid);
        count += mergeSort(nums, mid + 1, right);
        count += countPairs(nums, left, mid, right);

        // Standard merge process to keep the arrays sorted for the next layer
up
        merge(nums, left, mid, right);

        return count;
    }

    // The magical linear counting logic
    private int countPairs(int[] nums, int left, int mid, int right) {
        int count = 0;
        int j = mid + 1; // Start of right array

        for (int i = left; i <= mid; i++) {
```

```java
            // Traverse j linearly rightwards as long as the condition holds
            while (j <= right && (long)nums[i] > 2L * (long)nums[j]) {
                j++;
            }
            // All elements from (mid+1) up to (j-1) satisfy the condition!
            count += (j - (mid + 1));
        }
        return count;
    }

    // Standard Merge Sort Array merging
    private void merge(int[] nums, int left, int mid, int right) {
        int[] temp = new int[right - left + 1];
        int i = left, j = mid + 1, k = 0;

        while (i <= mid && j <= right) {
            if (nums[i] <= nums[j]) {
                temp[k++] = nums[i++];
            } else {
                temp[k++] = nums[j++];
            }
        }

        while (i <= mid) temp[k++] = nums[i++];
        while (j <= right) temp[k++] = nums[j++];

        for (int p = 0; p < temp.length; p++) {
            nums[left + p] = temp[p];
        }
    }
}
```

5. Dry Run & Architectural Insight

- **WHAT:** By injecting the `countPairs()` function between the `divide` recursive block and the `merge` execution block, we intercept the sorted sub-arrays when they are fully evaluated chronologically but haven't been spatially joined yet.
- **HOW:** Because the Left sub-array sits functionally *before* the Right sub-array chronologically, every interaction between them perfectly replicates $i < j$, yet because they are internally sorted, we can execute mathematical jump-counting `O(N)` instead of nested loops.
- **Result:** We solve an `O(N^2)` permutation problem natively at `O(N log N)` sorting speeds!

📝 DAILY SUMMARY

🎯 Patterns Learned Today

1. **Destructive Pointer Writes:** Understanding how to compact arrays natively in `O(1)` space by allowing a faster read pointer to override garbage data trailing behind a structurally safe write pointer.
2. **Dual-Bottleneck Constraint Validation:** Using bounding variables from both extremities of an array to mathematically seal interior volumetric potentials, rendering interior layout variations irrelevant (Trapping Rain Water).
3. **Divide & Conquer Data Interception:** Modifying standard foundational frameworks (Merge Sort) to execute analytical evaluation on fractional sorted states before memory compaction natively strips temporal indices.

⚖️ Decision Rules

- If given **"Remove Elements In-Place"** $=>$ Dedicate `Index 0` as the baseline. Put your Write Pointer at 0, your Scouter pointer at 1, and only Write when Scouter proves uniqueness.
- If asked about **Physical Volumes / Area Trapping** $=>$ Run absolute limits inward from `index[0]` and `index[N-1]` tracking `globalMaxes`.
- If asked about *Relational Pair Counts $(i < j \text{ and } A[i] > K$A[j]) `**` =>$ NEVER Two Pointer. Piggyback entirely onto Merge Sort Recursive divisions!

🏆 Week 1 Complete!

Congratulations. You are now wielding Array logic like an absolute Principal Engineer. You command mathematical bounds, Hash Frequency mappings, Combinatorics, and destructive pointer algorithms over arbitrary loops.

Next up, we will synthesize all of this logic into a single **PDF-Ready Revision Cheatsheet**!

Week 1 DSA Mastery: Revision Cheatsheet

Welcome to your End-of-Week Revision! This document synthesizes all the architectural patterns, algorithmic rules, and decision structures you've learned over the past 7 days. Review this sheet before any interview where Arrays, Hashing, or Pointers are involved.

---

1. The Ultimate Pattern Decision Tree

When confronted with an Array/Pointer problem in an interview, trace down this logic tree:

1. **Does the problem ask to find a specific Sum/Target (`K`) or XOR?**
   - **Are the numbers strictly positive?** $=>$ **Sliding Window** (Expand right, shrink left).
   - **Are there negative numbers?** $=>$ **Prefix Sum + HashMap** (`History = Current - K`).
   - **Does it ask for subsets/pairs of a specific size (2Sum, 4Sum)?** $=>$ Sort array, anchor loops, use **Two Pointers (Left/Right convergence)** to close the gap.
2. **Does the problem ask for "Continuous Subarrays" or "Substrings"?**
   - **"Longest without repeating characters"?** $=>$ **HashMap Teleporting Window** (`Math.max(L, oldIndex + 1)`).
   - **"Largest Sum (negatives included)"?** $=>$ **Kadane's Algorithm** (Aggressively reset running sum to `0` if it drops `< 0`).
3. **Does the problem involve "Modifying in-place", "Keeping relative order", or "Memory restrictions"?**
   - **"Sort EXACTLY 3 categories (0s, 1s, 2s) in-place"?** $=>$ **Dutch National Flag (Dijkstra's 3-Way Partition)**.
   - **"Remove Duplicates in-place"?** $=>$ **Slow & Fast Pointers** (Scout with Fast, overwrite with Slow).
   - **"Modify Matrix Zeroes in `O(1)` space"?** $=>$ **Dummy Caching** (Hijack the 1st row and 1st column as your boolean tracker arrays).
4. **Does the problem involve specific Mathematical Guarantees?**
   - **"Element appears `> N/2` or `> N/3` times"?** $=>$ **Moore's Voting Algorithm** (Mutual Annihilation).
   - **"Next Permutation / dictionary order"?** $=>$ **Lexical Suffix Peak Finding** (Find break `i < i+1` , swap minimal upgrade, reverse tail).
   - **"Count Paths in grid"?** $=>$ **Combinatorics Math (`nCr`)** (Destroy the 2D DP matrix!).

---

2. Core Array "Cheat Codes"

The Dual-Bottleneck Cheat Code (Rain Water)

To mathematically lock volumetric boundaries locally:

```
// If left is structurally weaker, it governs the maximum ceiling
if (leftWall <= rightWall) {
    if (current >= leftMax) leftMax = current;
    else water += leftMax - current;
    left++;
}
```

The In-Place Swap Cheat Code

Used universally in Two Pointers, Next Permutation, and DNF Sorting partitions:

```
private void swap(int[] arr, int i, int j) {
    int temp = arr[i];
    arr[i] = arr[j];
    arr[j] = temp;
}
```

The Duplicate Skip Cheat Code (K-Sum)

Whenever filtering permutations or unique combinations using Sorted Two Pointers:

```
while (left < right && nums[left] == nums[left - 1]) left++; // Skips adjacent
identicals
```

The Prefix HashMap Frequency Cheat Code

When counting total valid subarrays instead of just finding the longest:

```
// Initialize the mathematical zero-state before the loop!
map.put(0, 1);
```

```
// Update frequency without NullPointerExceptions
map.put(sum, map.getOrDefault(sum, 0) + 1);
```

---

3. Complexity Matrix

Core Pattern	General Time	General Space	Architectural Analogy
Two Pointers (Sorted Convergence)	`O(N log N)` (due to sort)	`O(1)`	Dual-Thread Validation
Sliding Window (Variable)	`O(N)`	`O(\text{Dictionary})`	TCP Packet Resets / Frame drops
Prefix Sum/XOR (Hash)	`O(N)`	`O(N)`	Kafka Stream Event Offsets
Kadane's Algorithm	`O(N)`	`O(1)`	Greedy Congestion Avoidance
Boyer-Moore's Voting	`O(N)`	`O(1)`	Byzantine Fault Tolerance Consensus
Dutch National Flag (3-Way)	`O(N)`	`O(1)`	Garbage Collection Partitioning
Prefix Sequence Hashing	`O(N)`	`O(N)`	Disjoint Graph Root Evaluation
Linear Algebra Transposition	`O(N^2)`	`O(1)`	GPU Texture Matrix Coordinate Swaps
Merge Sort Overriding	`O(N log N)`	`O(N)`	Spatial Pre-computation Interception

---

4. Senior Developer Traps (Pitfalls to Avoid)

1. **The Sliding Window on Negatives Trap**:
    - A Junior thinks: "I want a sum of 5. My current window sum is 6. I will shrink the left side to reduce the sum."
    - *Reality:* What if the left-side number you just removed was `-50`? By removing `-50`, your sum just skyrocketed to `56`! Monotonic windows ONLY work on positive integers. Always pivot to **Prefix Sum + HashMap** if negatives exist.
2. **The K-Sum Integer Overflow Trap**:

- FAANG testers will explicitly deploy arrays with `Integer.MAX_VALUE`. Adding four of them up in `4Sum` (`[a]+[b]+[c]+[d]`) instantly shatters the 32-bit `int` boundary into negative garbage.
- *Fix:* Always cast the execution block: `long sum = (long)nums[a] + nums[b] + nums[c] + nums[d];`

3. **The Matrix Scanning Desync**:
   - In *Set Matrix Zeroes*, if you mark your boolean triggers in the 0th Row and 0th Col during Phase 1... and then during Phase 2 you iterate from Top-Left to Bottom-Right, you will instantly overwrite your own trigger columns with `0` s before you ever read them.
   - *Fix:* Always iterate the mutation phase backwards (`Bottom-Right` $->$ `Top-Left`).
4. **The False Majority Assumption**:
   - In *Majority Element II* (*`>N/3`*), just because 2 elements survived the Boyer-Moore mutual annihilation phase does NOT mean they actually appear `>N/3` times. They might just be the "least attacked" minorities.
   - *Fix:* If the problem statement does not explicitly guarantee validity, a Phase-2 secondary assertion loop is strictly mandatory.

Final Week 1 Summary

You have evolved significantly. You no longer build nested `O(N^2)` loops. You now manipulate spatial memory limits natively in `O(1)`, execute algebraic complements dynamically against stream caches, and destroy dimensionality using `K`-Sum sorting tricks. You are now thinking like a Principal Engineer.

Get ready for Week 2! (Typically featuring Fast/Slow Pointers on Linked Lists, and Deep Binary Search).